DBMS Capstone Project

Name- Shreya Gupta
Roll No-2501420054
Btech CSE (Data Science)
Subject Name - DBMS
Faculty Name- Ms.Swati Gupta
Submission Date-10/11/2026

INDEX

S.NO.	Contents
1.	Introduction
2.	Objectives
3.	Database Design
4.	All Queries
5.	Tables and their constraints
6.	Creation of database and tables
7.	Insertion of sample records
8.	Inventory Analysis and Optimization
9.	Transportation Cost Optimization
10.	Order Fulfillment Analysis
11.	Supplier Performance Analysis
12.	Indexing and Query Optimization
13.	Composite Index on Inventory
14.	Composite Index on Purchase Orders
15.	Covering Index for Stock-out Report
16.	EXPLAIN Query Execution Analysis
17.	Creation of Low-Stock Alert View
18.	Creation of Supplier Scorecard View
19.	View Updatability Analysis
20.	Window Functions – ROW_NUMBER()
21.	Window Functions – SUM() OVER()
22.	Window Functions – LEAD()
23.	Screenshots of Outputs
24.	Conclusion

INTRODUCTION

Supply Chain Optimization is a database management project designed to improve the efficiency of inventory management, supplier coordination, transportation and order fulfillment. The system uses SQL queries, indexes, views and window functions to monitor inventory levels, identify stock shortages, analyze supplier performance and support better business decisions.

OBJECTIVES

- Design a relational database for supply chain management.
- Monitor inventory levels across warehouses.
- Identify products requiring replenishment.
- Analyze supplier delivery performance.
- Reduce transportation costs.
- Improve order fulfillment.
- Use indexes and advanced SQL features to optimize queries.

PROJECT NO-5
Supply Chain Optimization

All queries:-

```
CREATE DATABASE supply_chain_optimization;
USE supply_chain_optimization;
CREATE TABLE suppliers (
   supplier_id INT AUTO_INCREMENT PRIMARY KEY,
   supplier_name VARCHAR(100) NOT NULL,
   contact_person VARCHAR(100),
   phone VARCHAR(15),
   email VARCHAR(100) UNIQUE,
   city VARCHAR(50),
   supplier_rating DECIMAL(3,2)
);
CREATE TABLE manufacturers (
   manufacturer_id INT AUTO_INCREMENT PRIMARY KEY,
   manufacturer_name VARCHAR(100) NOT NULL,
   city VARCHAR(50),
   contact_email VARCHAR(100) UNIQUE
);
CREATE TABLE distributors (
   distributor_id INT AUTO_INCREMENT PRIMARY KEY,
   distributor_name VARCHAR(100) NOT NULL,
   city VARCHAR(50),
   phone VARCHAR(15)
);
CREATE TABLE products (
   product_id INT AUTO_INCREMENT PRIMARY KEY,
   product_name VARCHAR(100) NOT NULL,
   manufacturer_id INT,
   unit_price DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (manufacturer_id)
       REFERENCES manufacturers(manufacturer_id),
   CHECK (unit_price >= 0)
);
CREATE TABLE warehouses (
   warehouse_id INT AUTO_INCREMENT PRIMARY KEY,
   warehouse_name VARCHAR(100) NOT NULL,
```

```
    location VARCHAR(100),
    capacity INT NOT NULL,
    CHECK (capacity > 0)
);
CREATE TABLE inventory (
    inventory_id INT AUTO_INCREMENT PRIMARY KEY,
    warehouse_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity_on_hand INT NOT NULL DEFAULT 0,
    reorder_level INT NOT NULL,
    last_updated DATE,
    FOREIGN KEY (warehouse_id)
        REFERENCES warehouses(warehouse_id),
    FOREIGN KEY (product_id)
        REFERENCES products(product_id),
    UNIQUE (warehouse_id, product_id),
    CHECK (quantity_on_hand >= 0),
    CHECK (reorder_level >= 0)
);
CREATE TABLE purchase_orders (
    po_id INT AUTO_INCREMENT PRIMARY KEY,
    supplier_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    order_date DATE NOT NULL,
    expected_delivery_date DATE,
    actual_delivery_date DATE,
    order_status VARCHAR(30) NOT NULL,
    FOREIGN KEY (supplier_id)
        REFERENCES suppliers(supplier_id),
    FOREIGN KEY (warehouse_id)
        REFERENCES warehouses(warehouse_id)
);
CREATE TABLE purchase_order_items (
    po_item_id INT AUTO_INCREMENT PRIMARY KEY,
    po_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity_ordered INT NOT NULL,
    quantity_received INT DEFAULT 0,
    unit_cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (po_id)
        REFERENCES purchase_orders(po_id),
    FOREIGN KEY (product_id)
        REFERENCES products(product_id),
    CHECK (quantity_ordered > 0),
    CHECK (quantity_received >= 0),
```

```
    CHECK (unit_cost >= 0)
);
CREATE TABLE inventory_movements (
    movement_id INT AUTO_INCREMENT PRIMARY KEY,
    warehouse_id INT NOT NULL,
    product_id INT NOT NULL,
    movement_type VARCHAR(20) NOT NULL,
    quantity INT NOT NULL,
    movement_date DATE NOT NULL,
    FOREIGN KEY (warehouse_id)
        REFERENCES warehouses(warehouse_id),
    FOREIGN KEY (product_id)
        REFERENCES products(product_id),
    CHECK (quantity > 0)
);
CREATE TABLE shipments (
    shipment_id INT AUTO_INCREMENT PRIMARY KEY,
    po_id INT NOT NULL,
    distributor_id INT,
    shipment_date DATE,
    delivery_date DATE,
    transportation_cost DECIMAL(10,2),
    shipment_status VARCHAR(30),
    FOREIGN KEY (po_id)
        REFERENCES purchase_orders(po_id),
    FOREIGN KEY (distributor_id)
        REFERENCES distributors(distributor_id),
    CHECK (transportation_cost >= 0)
);
SHOW TABLES;
INSERT INTO suppliers
(supplier_name, contact_person, phone, email, city, supplier_rating)
VALUES
('ABC Components', 'Rahul Sharma', '9876543210', 'abc@gmail.com', 'Delhi', 4.50),
('Global Supplies', 'Amit Verma', '9876543211', 'global@gmail.com', 'Mumbai',
4.20),
('Prime Distributors', 'Neha Kapoor', '9876543212', 'prime@gmail.com', 'Pune',
4.70),
('Reliable Traders', 'Karan Mehta', '9876543213', 'reliable@gmail.com', 'Jaipur',
3.90);
INSERT INTO manufacturers
(manufacturer_name, city, contact_email)
VALUES
('Tech Manufacturing Ltd', 'Delhi', 'tech@gmail.com'),
('Smart Industries', 'Mumbai', 'smart@gmail.com'),
```

```
('Advanced Products Co', 'Pune', 'advanced@gmail.com');
INSERT INTO distributors
(distributor_name, city, phone)
VALUES
('FastTrack Logistics', 'Delhi', '9811111111'),
('Express Delivery', 'Mumbai', '9822222222'),
('QuickMove Transport', 'Pune', '9833333333');
INSERT INTO products
(product_name, manufacturer_id, unit_price)
VALUES
('Laptop', 1, 55000.00),
('Wireless Mouse', 2, 800.00),
('Keyboard', 2, 1500.00),
('Monitor', 3, 12000.00),
('Headphones', 1, 2500.00);
INSERT INTO warehouses
(warehouse_name, location, capacity)
VALUES
('Central Warehouse', 'Delhi', 10000),
('Western Warehouse', 'Mumbai', 8000),
('Southern Warehouse', 'Pune', 6000);
INSERT INTO inventory
(warehouse_id, product_id, quantity_on_hand, reorder_level, last_updated)
VALUES
(1, 1, 20, 30, '2026-11-01'),
(1, 2, 150, 50, '2026-11-01'),
(1, 3, 25, 40, '2026-11-01'),
(2, 1, 15, 25, '2026-11-02'),
(2, 4, 60, 30, '2026-11-02'),
(2, 5, 10, 20, '2026-11-02'),
(3, 2, 35, 40, '2026-11-03'),
(3, 3, 80, 30, '2026-11-03');
INSERT INTO purchase_orders
(supplier_id, warehouse_id, order_date, expected_delivery_date,
actual_delivery_date, order_status)
VALUES
(1, 1, '2026-10-20', '2026-10-25', '2026-10-24', 'Delivered'),
(2, 1, '2026-10-28', '2026-11-05', NULL, 'Pending'),
(3, 2, '2026-10-22', '2026-10-29', '2026-10-30', 'Delivered'),
(4, 2, '2026-11-01', '2026-11-10', NULL, 'In Transit'),
(1, 3, '2026-11-02', '2026-11-12', NULL, 'Pending');
INSERT INTO purchase_order_items
(po_id, product_id, quantity_ordered, quantity_received, unit_cost)
VALUES
(1, 1, 50, 50, 50000.00),
```

```
(1, 2, 100, 100, 700.00),
(2, 3, 80, 0, 1300.00),
(3, 4, 40, 40, 11000.00),
(4, 5, 60, 30, 2200.00),
(5, 2, 100, 0, 700.00),
(5, 3, 50, 0, 1300.00);
INSERT INTO inventory_movements
(warehouse_id, product_id, movement_type, quantity, movement_date)
VALUES
(1, 1, 'IN', 50, '2026-10-20'),
(1, 1, 'OUT', 20, '2026-10-22'),
(1, 1, 'OUT', 10, '2026-10-25'),
(1, 2, 'IN', 100, '2026-10-21'),
(1, 2, 'OUT', 30, '2026-10-23'),
(2, 4, 'IN', 40, '2026-10-22'),
(2, 4, 'OUT', 15, '2026-10-25'),
(2, 5, 'OUT', 10, '2026-11-02');
INSERT INTO shipments
(po_id, distributor_id, shipment_date, delivery_date,
transportation_cost, shipment_status)
VALUES
(1, 1, '2026-10-20', '2026-10-24', 2500.00, 'Delivered'),
(2, 2, '2026-10-29', NULL, 1800.00, 'In Transit'),
(3, 3, '2026-10-23', '2026-10-30', 2200.00, 'Delivered'),
(4, 1, '2026-11-02', NULL, 1500.00, 'In Transit'),
(5, 2, '2026-11-03', NULL, 2000.00, 'Pending');
SELECT 'suppliers' AS table_name, COUNT(*) AS total_records FROM suppliers
UNION ALL
SELECT 'manufacturers', COUNT(*) FROM manufacturers
UNION ALL
SELECT 'distributors', COUNT(*) FROM distributors
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'warehouses', COUNT(*) FROM warehouses
UNION ALL
SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL
SELECT 'purchase_orders', COUNT(*) FROM purchase_orders
UNION ALL
SELECT 'purchase_order_items', COUNT(*) FROM purchase_order_items
UNION ALL
SELECT 'inventory_movements', COUNT(*) FROM inventory_movements
UNION ALL
SELECT 'shipments', COUNT(*) FROM shipments;
```

```sql
# Display complete inventory details
SELECT
   w.warehouse_name,
   p.product_name,
   i.quantity_on_hand,
   i.reorder_level,
   i.last_updated
FROM inventory i
JOIN warehouses w
   ON i.warehouse_id = w.warehouse_id
JOIN products p
   ON i.product_id = p.product_id
ORDER BY w.warehouse_id, p.product_id;
# Identify products below their reorder level
SELECT
   w.warehouse_name,
   p.product_name,
   i.quantity_on_hand,
   i.reorder_level
FROM inventory i
JOIN warehouses w
   ON i.warehouse_id = w.warehouse_id
JOIN products p
   ON i.product_id = p.product_id
WHERE i.quantity_on_hand < i.reorder_level
ORDER BY i.quantity_on_hand;
#Calculate pending quantities from purchase orders
SELECT
   po.po_id,
   s.supplier_name,
   p.product_name,
   po.order_status,
   poi.quantity_ordered,
   poi.quantity_received,
   (poi.quantity_ordered - poi.quantity_received) AS pending_quantity
FROM purchase_orders po
JOIN suppliers s
   ON po.supplier_id = s.supplier_id
JOIN purchase_order_items poi
   ON po.po_id = poi.po_id
JOIN products p
   ON poi.product_id = p.product_id
WHERE po.order_status IN ('Pending', 'In Transit')
 AND poi.quantity_ordered > poi.quantity_received
ORDER BY po.po_id;
```

```sql
# Calculate total inventory value in each warehouse
SELECT
   w.warehouse_name,
   SUM(i.quantity_on_hand) AS total_units,
   SUM(i.quantity_on_hand * p.unit_price) AS total_inventory_value
FROM inventory i
JOIN warehouses w
   ON i.warehouse_id = w.warehouse_id
JOIN products p
   ON i.product_id = p.product_id
GROUP BY w.warehouse_id, w.warehouse_name
ORDER BY total_inventory_value DESC;
# Calculate the recommended replenishment quantity
SELECT
   w.warehouse_name,
   p.product_name,
   i.quantity_on_hand,
   i.reorder_level,
   GREATEST(i.reorder_level - i.quantity_on_hand, 0)
       AS recommended_replenishment
FROM inventory i
JOIN warehouses w
   ON i.warehouse_id = w.warehouse_id
JOIN products p
   ON i.product_id = p.product_id
ORDER BY w.warehouse_id, p.product_id;

#reducing transportation costs and improving order fulfillment

#Calculate total transportation cost by distributor
SELECT
   d.distributor_name,
   COUNT(s.shipment_id) AS total_shipments,
   SUM(s.transportation_cost) AS total_transportation_cost
FROM shipments s
JOIN distributors d
   ON s.distributor_id = d.distributor_id
GROUP BY d.distributor_id, d.distributor_name
ORDER BY total_transportation_cost DESC;

#Identify the most expensive shipments
SELECT
   s.shipment_id,
   d.distributor_name,
   s.po_id,
```

```
    s.transportation_cost,
    s.shipment_status
FROM shipments s
JOIN distributors d
    ON s.distributor_id = d.distributor_id
ORDER BY s.transportation_cost DESC;

# Calculate total transportation expenditure
SELECT
    COUNT(*) AS total_shipments,
    SUM(transportation_cost) AS total_transportation_cost,
    AVG(transportation_cost) AS average_transportation_cost
FROM shipments;

# Order fulfillment and supplier performance

#Display purchase order fulfillment status

SELECT
    po.po_id,
    s.supplier_name,
    po.order_date,
    po.expected_delivery_date,
    po.actual_delivery_date,
    po.order_status
FROM purchase_orders po
JOIN suppliers s
    ON po.supplier_id = s.supplier_id
ORDER BY po.po_id;

# Calculate supplier on-time delivery percentage
SELECT
    s.supplier_name,
    COUNT(po.po_id) AS delivered_orders,
    SUM(
        CASE
            WHEN po.actual_delivery_date <= po.expected_delivery_date
            THEN 1
            ELSE 0
        END
    ) AS on_time_orders,
    ROUND(
        100.0 * SUM(
            CASE
                WHEN po.actual_delivery_date <= po.expected_delivery_date
```

```
                THEN 1
                ELSE 0
            END
        ) / COUNT(po.po_id),
        2
    ) AS on_time_delivery_percentage
FROM suppliers s
JOIN purchase_orders po
    ON s.supplier_id = po.supplier_id
WHERE po.order_status = 'Delivered'
GROUP BY s.supplier_id, s.supplier_name
ORDER BY on_time_delivery_percentage DESC;

#Calculate average supplier delivery lead time
SELECT
    s.supplier_name,
    COUNT(po.po_id) AS delivered_orders,
    ROUND(
        AVG(DATEDIFF(
            po.actual_delivery_date,
            po.order_date
        )),
        2
    ) AS average_lead_time_days
FROM suppliers s
JOIN purchase_orders po
    ON s.supplier_id = po.supplier_id
WHERE po.actual_delivery_date IS NOT NULL
GROUP BY s.supplier_id, s.supplier_name
ORDER BY average_lead_time_days ASC;

#Identify pending order quantities
SELECT
    po.po_id,
    s.supplier_name,
    p.product_name,
    poi.quantity_ordered,
    poi.quantity_received,
    (poi.quantity_ordered - poi.quantity_received)
        AS remaining_quantity,
    po.order_status
FROM purchase_orders po
JOIN suppliers s
    ON po.supplier_id = s.supplier_id
JOIN purchase_order_items poi
```

```
   ON po.po_id = poi.po_id
JOIN products p
   ON poi.product_id = p.product_id
WHERE poi.quantity_received < poi.quantity_ordered
ORDER BY remaining_quantity DESC;

# Calculate overall order fulfillment percentage

SELECT
   SUM(quantity_ordered) AS total_units_ordered,
   SUM(quantity_received) AS total_units_received,
   ROUND(
       100.0 * SUM(quantity_received) /
       SUM(quantity_ordered),
       2
   ) AS fulfillment_percentage
FROM purchase_order_items;

# Indexing and query optimization

#Check query execution plan before indexing
EXPLAIN
SELECT
   warehouse_id,
   product_id,
   quantity_on_hand,
   reorder_level
FROM inventory
WHERE quantity_on_hand < reorder_level;

#Create a composite index on inventory
CREATE INDEX idx_inventory_warehouse_product
ON inventory (warehouse_id, product_id);

# Create a composite index on purchase orders
CREATE INDEX idx_po_supplier_status_delivery
ON purchase_orders
(supplier_id, order_status, expected_delivery_date);

# Create a covering index for the stock-out report
CREATE INDEX idx_inventory_stockout_report
ON inventory
(warehouse_id, product_id, quantity_on_hand, reorder_level);

#Display the indexes created
```

```sql
SHOW INDEX FROM inventory;

#Display purchase order indexes
SHOW INDEX FROM purchase_orders;

#Check query execution plan after indexing
EXPLAIN
SELECT
   warehouse_id,
   product_id,
   quantity_on_hand,
   reorder_level
FROM inventory
WHERE quantity_on_hand < reorder_level;

#Create and use views

# Create a low-stock alert view

CREATE OR REPLACE VIEW vw_low_stock_alert AS
SELECT
   w.warehouse_name,
   p.product_name,
   i.quantity_on_hand,
   i.reorder_level,
   COALESCE(
       (
           SELECT SUM(
               poi.quantity_ordered - poi.quantity_received
           )
           FROM purchase_orders po
           JOIN purchase_order_items poi
               ON po.po_id = poi.po_id
           WHERE po.warehouse_id = i.warehouse_id
             AND poi.product_id = i.product_id
             AND po.order_status IN ('Pending', 'In Transit')
             AND poi.quantity_ordered > poi.quantity_received
       ),
       0
   ) AS pending_po_quantity,
   i.quantity_on_hand +
   COALESCE(
       (
           SELECT SUM(
               poi.quantity_ordered - poi.quantity_received
```

```
          )
          FROM purchase_orders po
          JOIN purchase_order_items poi
              ON po.po_id = poi.po_id
          WHERE po.warehouse_id = i.warehouse_id
            AND poi.product_id = i.product_id
            AND po.order_status IN ('Pending', 'In Transit')
            AND poi.quantity_ordered > poi.quantity_received
      ),
      0
   ) AS projected_stock
FROM inventory i
JOIN warehouses w
   ON i.warehouse_id = w.warehouse_id
JOIN products p
   ON i.product_id = p.product_id
WHERE i.quantity_on_hand < i.reorder_level;

# Display the low-stock alert report

SELECT *
FROM vw_low_stock_alert
ORDER BY warehouse_name, product_name;

#Create the supplier scorecard view

CREATE OR REPLACE VIEW vw_supplier_scorecard AS
SELECT
   s.supplier_id,
   s.supplier_name,

   COALESCE(ds.total_delivered_orders, 0)
       AS total_delivered_orders,

   COALESCE(ds.on_time_orders, 0)
       AS on_time_orders,

   COALESCE(
       ROUND(
           100.0 * ds.on_time_orders /
           NULLIF(ds.total_delivered_orders, 0),
           2
       ),
       0
   ) AS on_time_delivery_percentage,
```

```
    COALESCE(ds.average_lead_time_days, 0)
        AS average_lead_time_days,

    COALESCE(ls.total_order_lines, 0)
        AS total_order_lines,

    COALESCE(ls.short_received_lines, 0)
        AS short_received_lines,

    COALESCE(
        ROUND(
            100.0 * ls.short_received_lines /
            NULLIF(ls.total_order_lines, 0),
            2
        ),
        0
    ) AS short_shipment_rate_percentage

FROM suppliers s

LEFT JOIN (
    SELECT
        supplier_id,
        COUNT(*) AS total_delivered_orders,
        SUM(
            CASE
                WHEN actual_delivery_date <= expected_delivery_date
                THEN 1 ELSE 0
            END
        ) AS on_time_orders,
        ROUND(
            AVG(
                DATEDIFF(actual_delivery_date, order_date)
            ),
            2
        ) AS average_lead_time_days
    FROM purchase_orders
    WHERE order_status = 'Delivered'
      AND actual_delivery_date IS NOT NULL
    GROUP BY supplier_id
) ds
    ON s.supplier_id = ds.supplier_id

LEFT JOIN (
```

```
    SELECT
        po.supplier_id,
        COUNT(*) AS total_order_lines,
        SUM(
            CASE
                WHEN poi.quantity_received < poi.quantity_ordered
                THEN 1 ELSE 0
            END
        ) AS short_received_lines
    FROM purchase_orders po
    JOIN purchase_order_items poi
        ON po.po_id = poi.po_id
    GROUP BY po.supplier_id
) ls
    ON s.supplier_id = ls.supplier_id;

# Display supplier scorecards
SELECT *
FROM vw_supplier_scorecard
ORDER BY on_time_delivery_percentage DESC;

# Check whether the views are updatable
SELECT
    TABLE_NAME,
    IS_UPDATABLE
FROM information_schema.VIEWS
WHERE TABLE_SCHEMA = DATABASE()
 AND TABLE_NAME IN (
     'vw_low_stock_alert',
     'vw_supplier_scorecard'
 );

# Rank primary and backup suppliers for each product
SELECT
    product_name,
    supplier_name,
    average_lead_time_days,
    average_unit_cost,
    supplier_rank,
    CASE
        WHEN supplier_rank = 1 THEN 'Primary Supplier'
        WHEN supplier_rank = 2 THEN 'Backup Supplier'
        ELSE 'Other Supplier'
    END AS supplier_category
FROM (
```

```sql
    SELECT
        product_id,
        product_name,
        supplier_name,
        average_lead_time_days,
        average_unit_cost,
        ROW_NUMBER() OVER (
            PARTITION BY product_id
            ORDER BY average_lead_time_days, average_unit_cost
        ) AS supplier_rank
    FROM (
        SELECT
            p.product_id,
            p.product_name,
            s.supplier_name,
            AVG(
                DATEDIFF(po.actual_delivery_date, po.order_date)
            ) AS average_lead_time_days,
            AVG(poi.unit_cost) AS average_unit_cost
        FROM purchase_orders po
        JOIN purchase_order_items poi
            ON po.po_id = poi.po_id
        JOIN suppliers s
            ON po.supplier_id = s.supplier_id
        JOIN products p
            ON poi.product_id = p.product_id
        WHERE po.order_status = 'Delivered'
          AND po.actual_delivery_date IS NOT NULL
        GROUP BY
            p.product_id,
            p.product_name,
            s.supplier_id,
            s.supplier_name
    ) supplier_metrics
) ranked_suppliers
WHERE supplier_rank <= 2
ORDER BY product_name, supplier_rank;

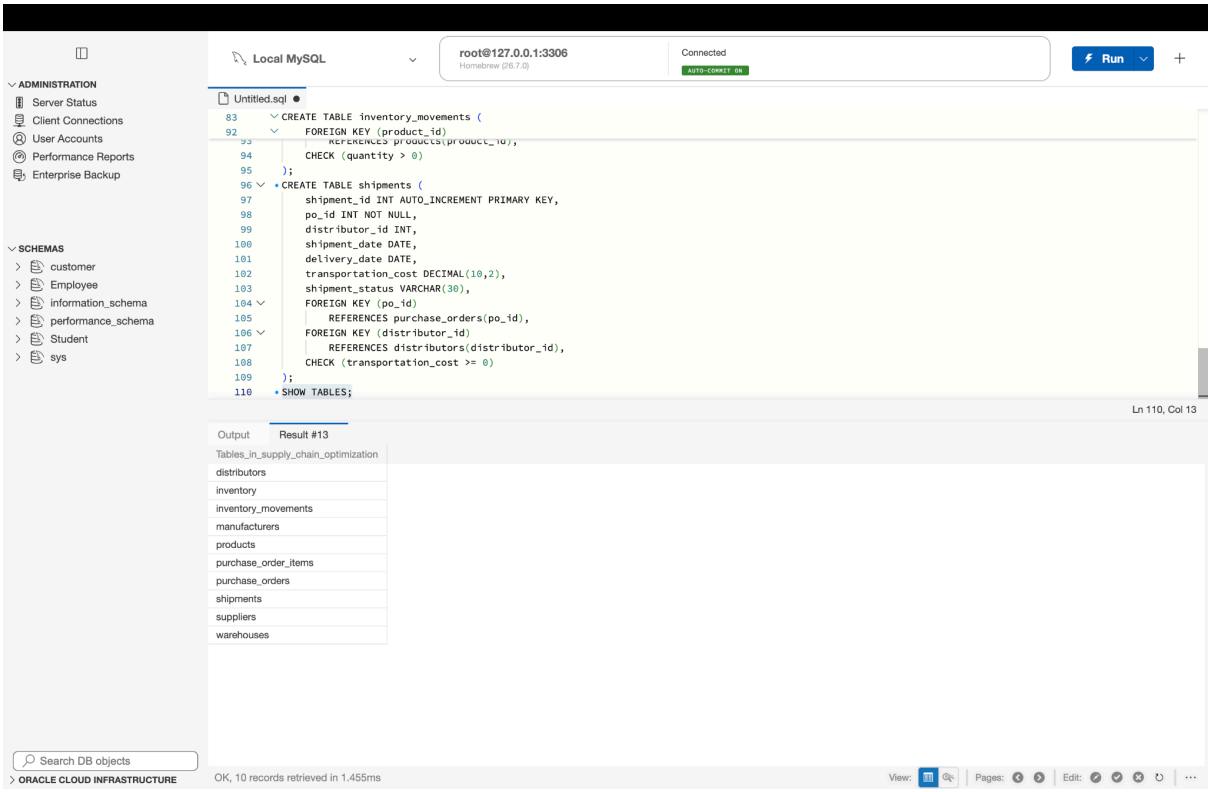
#Calculate running inventory levels using SUM() OVER()
WITH movement_balances AS (
    SELECT
        movement_id,
        warehouse_id,
        product_id,
        movement_date,
```

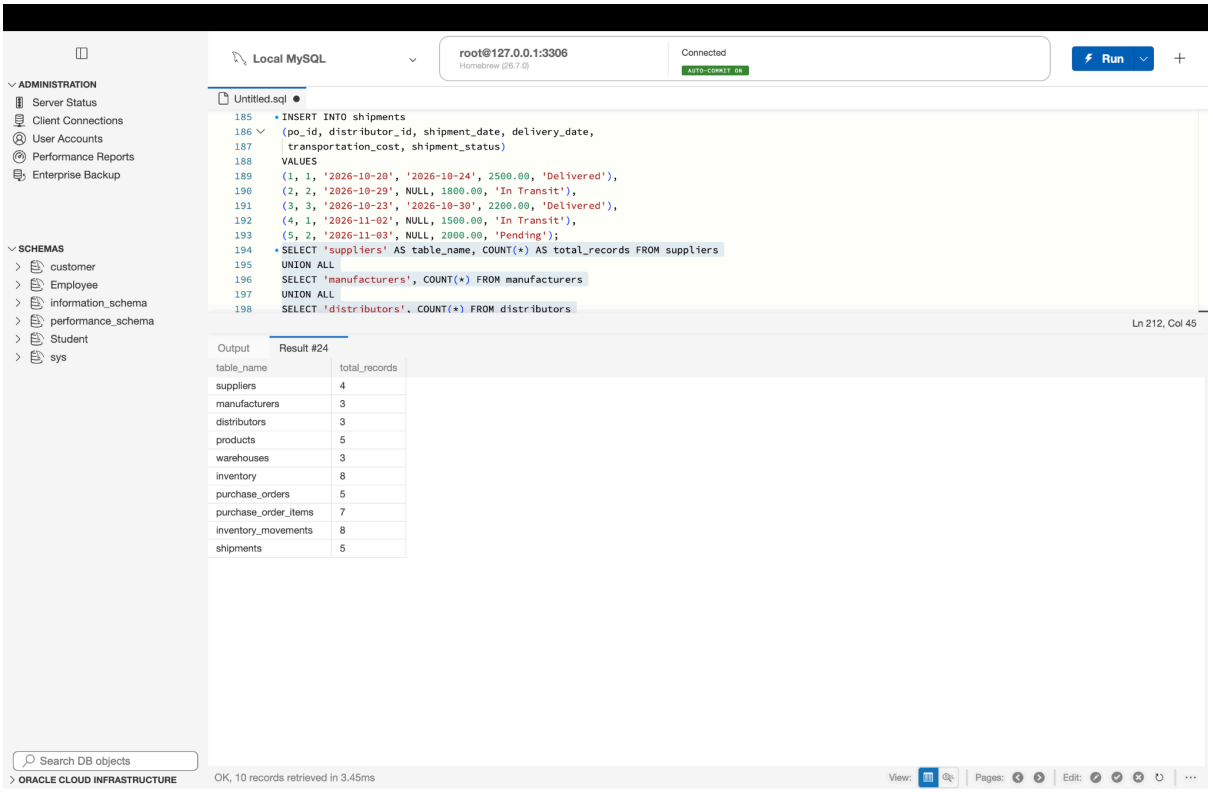
```
        CASE
            WHEN movement_type = 'IN' THEN quantity
            ELSE -quantity
        END AS net_change
    FROM inventory_movements
),
opening_balances AS (
    SELECT
        i.warehouse_id,
        i.product_id,
        i.quantity_on_hand -
            COALESCE(SUM(m.net_change), 0) AS opening_quantity
    FROM inventory i
    LEFT JOIN movement_balances m
        ON i.warehouse_id = m.warehouse_id
       AND i.product_id = m.product_id
    GROUP BY
        i.warehouse_id,
        i.product_id,
        i.quantity_on_hand
)
SELECT
    w.warehouse_name,
    p.product_name,
    m.movement_date,
    m.net_change,
    ob.opening_quantity +
        SUM(m.net_change) OVER (
            PARTITION BY m.warehouse_id, m.product_id
            ORDER BY m.movement_date, m.movement_id
            ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
        ) AS running_stock
FROM movement_balances m
JOIN opening_balances ob
    ON m.warehouse_id = ob.warehouse_id
   AND m.product_id = ob.product_id
JOIN warehouses w
    ON m.warehouse_id = w.warehouse_id
JOIN products p
    ON m.product_id = p.product_id
ORDER BY
    w.warehouse_name,
    p.product_name,
    m.movement_date;
```

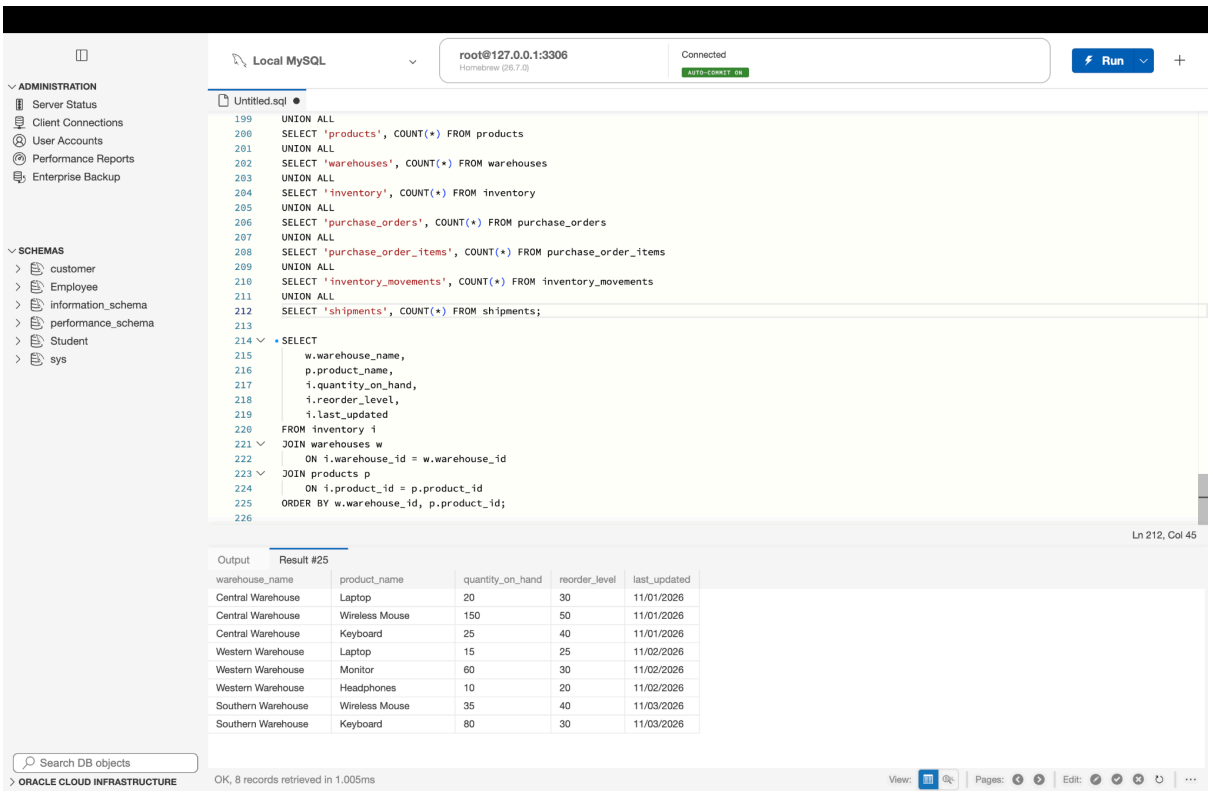
```sql
#Identify stock-out dates
WITH movement_balances AS (
   SELECT
       movement_id,
       warehouse_id,
       product_id,
       movement_date,
       CASE
           WHEN movement_type = 'IN' THEN quantity
           ELSE -quantity
       END AS net_change
   FROM inventory_movements
),
opening_balances AS (
   SELECT
       i.warehouse_id,
       i.product_id,
       i.quantity_on_hand -
           COALESCE(SUM(m.net_change), 0) AS opening_quantity
   FROM inventory i
   LEFT JOIN movement_balances m
       ON i.warehouse_id = m.warehouse_id
      AND i.product_id = m.product_id
   GROUP BY
       i.warehouse_id,
       i.product_id,
       i.quantity_on_hand
),
running_balances AS (
   SELECT
       m.warehouse_id,
       m.product_id,
       m.movement_date,
       ob.opening_quantity +
           SUM(m.net_change) OVER (
               PARTITION BY m.warehouse_id, m.product_id
               ORDER BY m.movement_date, m.movement_id
               ROWS BETWEEN UNBOUNDED PRECEDING
               AND CURRENT ROW
           ) AS running_stock
   FROM movement_balances m
   JOIN opening_balances ob
       ON m.warehouse_id = ob.warehouse_id
      AND m.product_id = ob.product_id
```

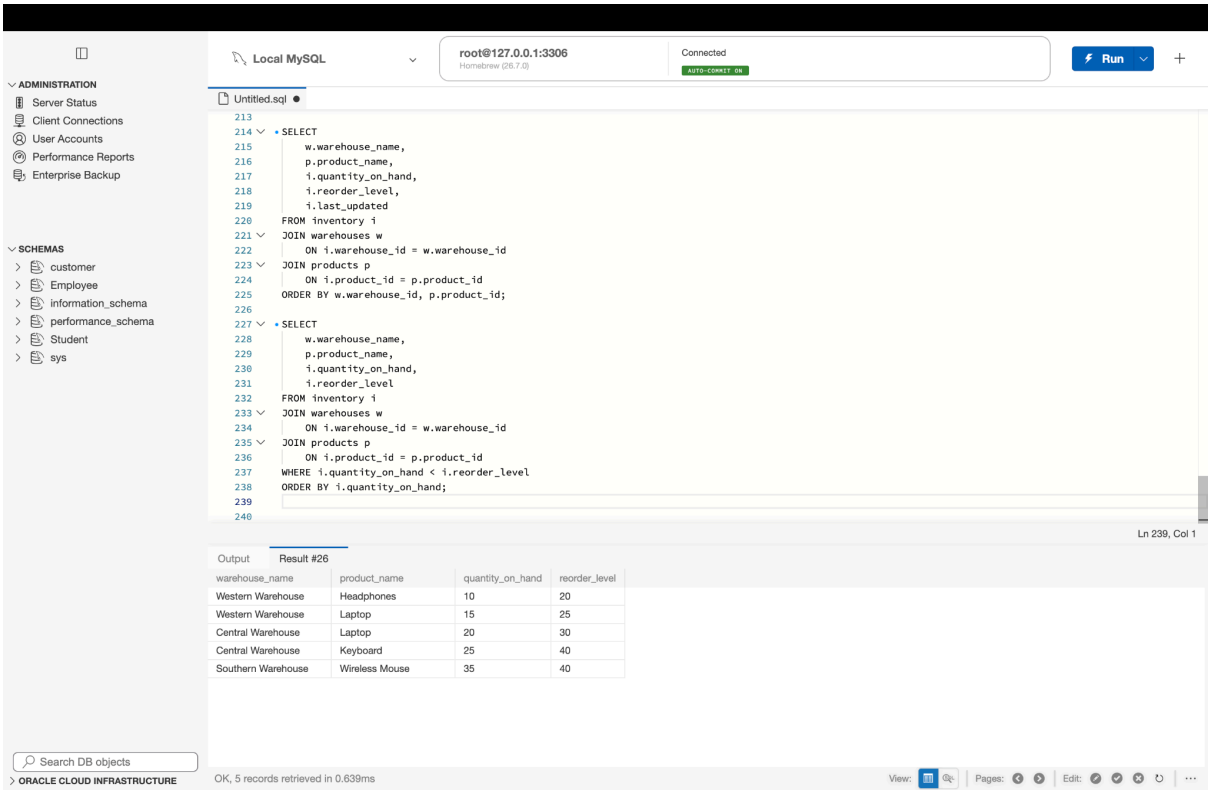
```sql
)
SELECT
   w.warehouse_name,
   p.product_name,
   MIN(
       CASE
           WHEN rb.running_stock <= 0
           THEN rb.movement_date
       END
   ) AS stockout_date
FROM running_balances rb
JOIN warehouses w
   ON rb.warehouse_id = w.warehouse_id
JOIN products p
   ON rb.product_id = p.product_id
GROUP BY
   rb.warehouse_id,
   rb.product_id,
   w.warehouse_name,
   p.product_name
HAVING stockout_date IS NOT NULL;
# Calculate days between consecutive replenishments using LEAD()
SELECT
   s.supplier_name,
   po.po_id,
   po.expected_delivery_date AS current_replenishment_date,
   LEAD(po.expected_delivery_date) OVER (
       PARTITION BY po.supplier_id
       ORDER BY po.expected_delivery_date
   ) AS next_replenishment_date,
   DATEDIFF(
       LEAD(po.expected_delivery_date) OVER (
           PARTITION BY po.supplier_id
           ORDER BY po.expected_delivery_date
       ),
       po.expected_delivery_date
   ) AS days_between_replenishments
FROM purchase_orders po
JOIN suppliers s
   ON po.supplier_id = s.supplier_id
ORDER BY
   s.supplier_name,
   po.expected_delivery_date;
```

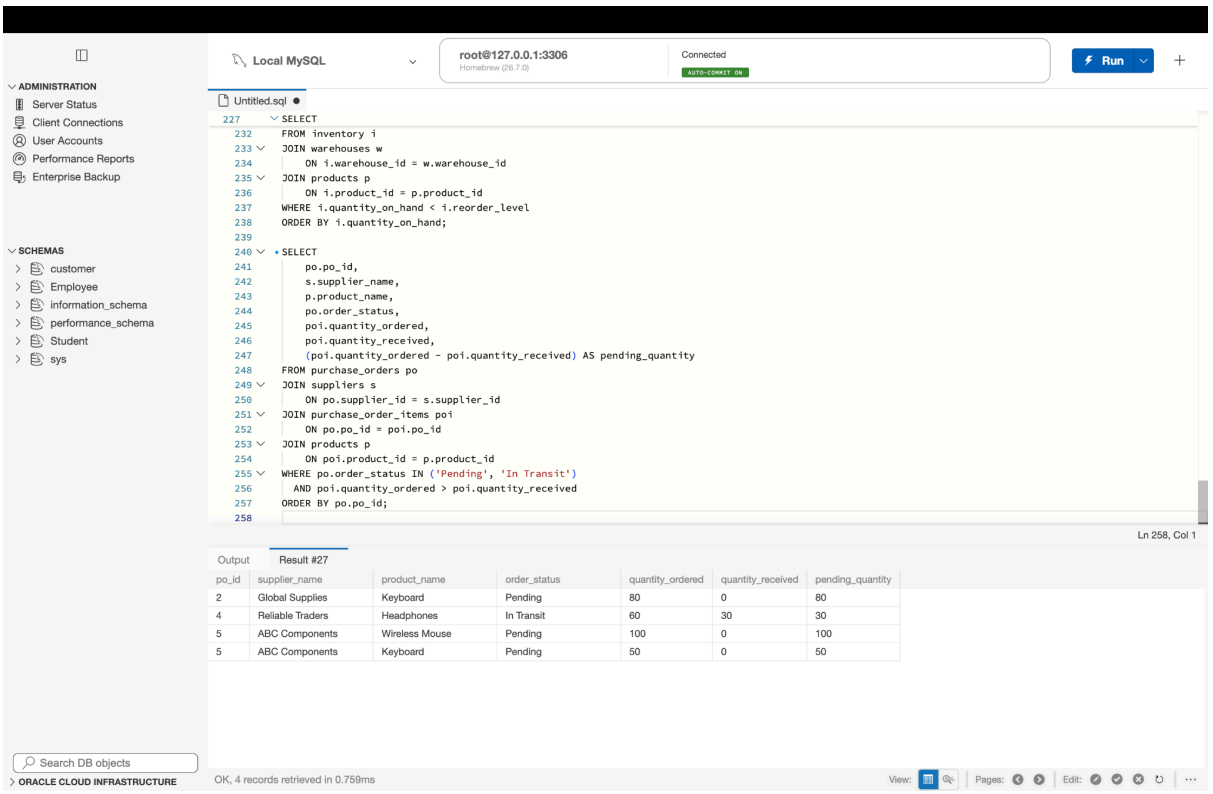
Screenshots with output:-

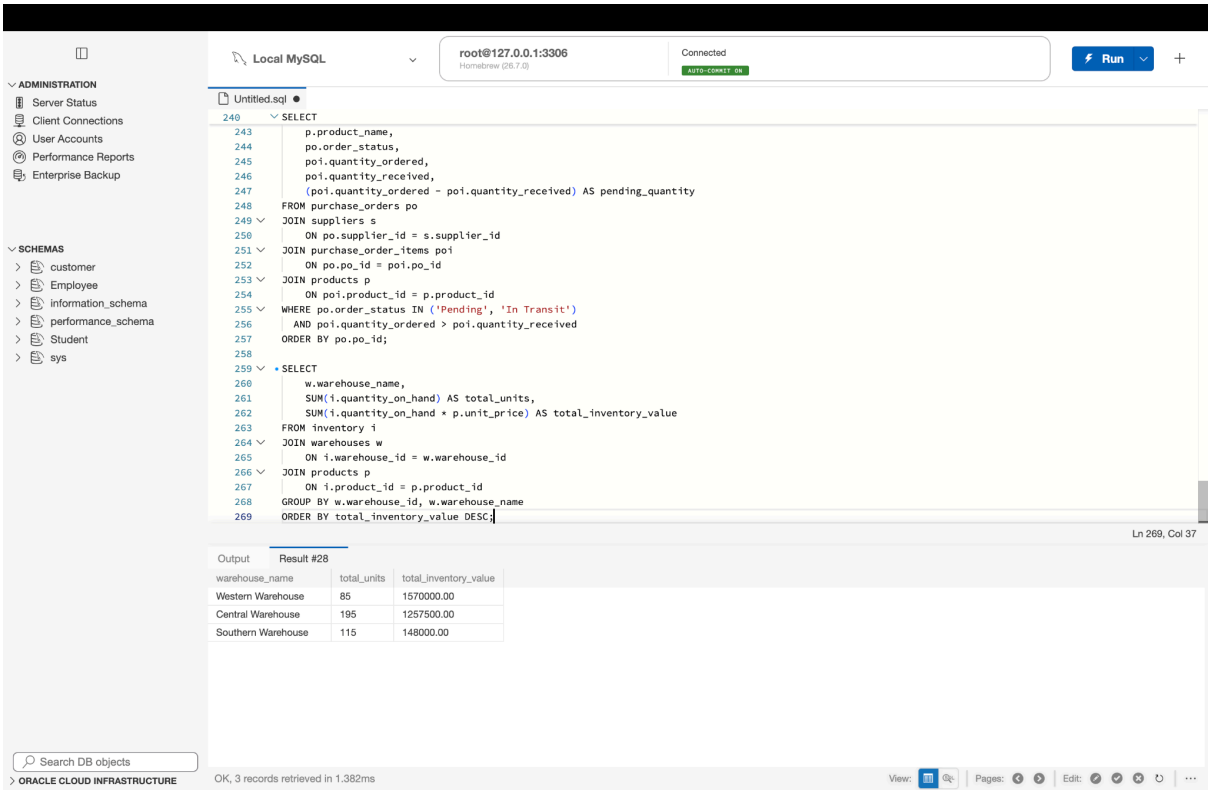


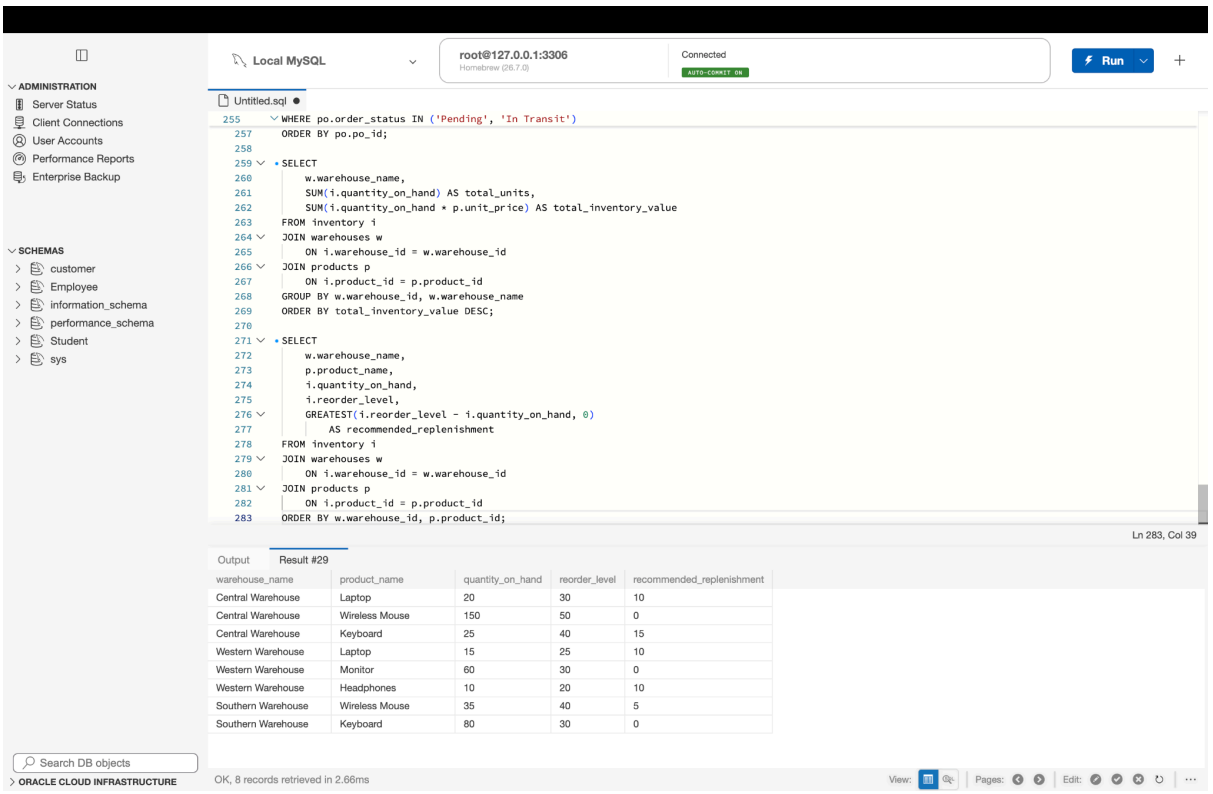


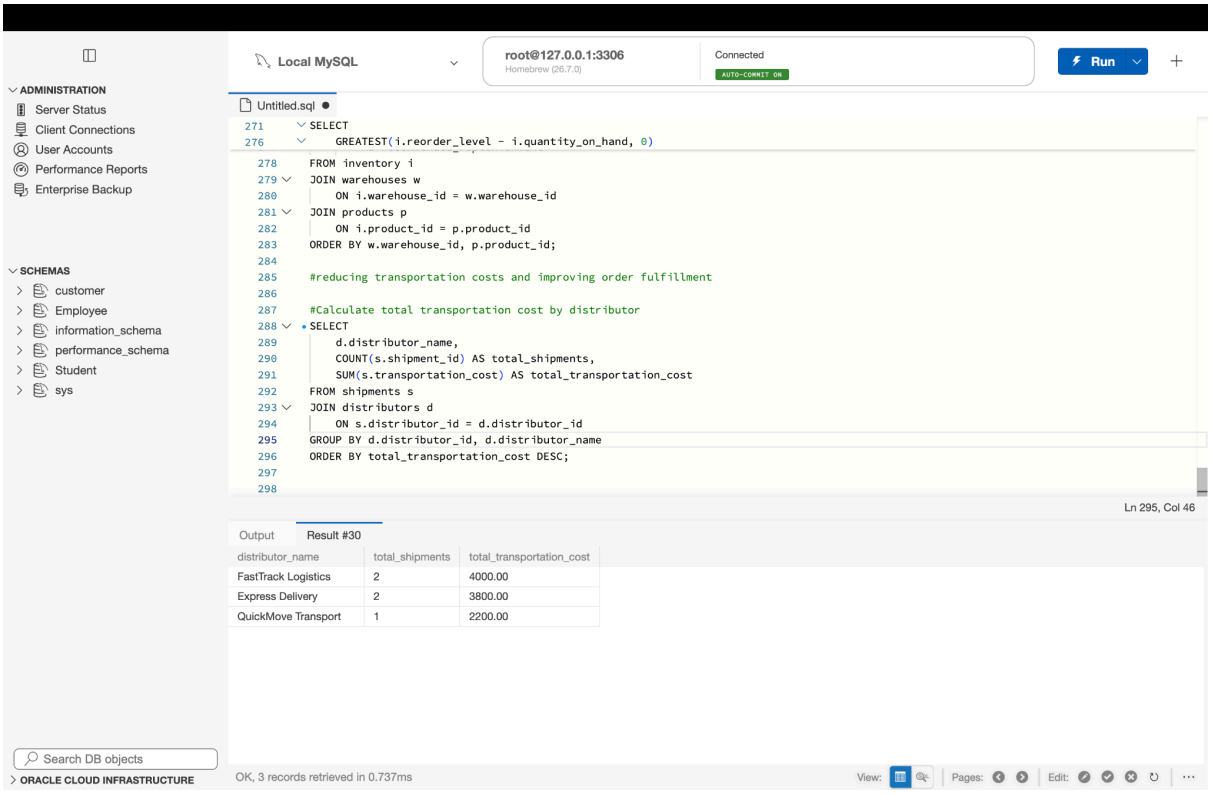

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
199 UNION ALL
200 SELECT 'products', COUNT(*) FROM products
201 UNION ALL
202 SELECT 'warehouses', COUNT(*) FROM warehouses
203 UNION ALL
204 SELECT 'inventory', COUNT(*) FROM inventory
205 UNION ALL
206 SELECT 'purchase_orders', COUNT(*) FROM purchase_orders
207 UNION ALL
208 SELECT 'purchase_order_items', COUNT(*) FROM purchase_order_items
209 UNION ALL
210 SELECT 'inventory_movements', COUNT(*) FROM inventory_movements
211 UNION ALL
212 SELECT 'shipments', COUNT(*) FROM shipments;
213
214 SELECT
215 w.warehouse_name,
216 p.product_name,
217 i.quantity_on_hand,
218 i.reorder_level,
219 i.last_updated
220 FROM inventory i
221 JOIN warehouses w
222 ON i.warehouse_id = w.warehouse_id
223 JOIN products p
224 ON i.product_id = p.product_id
225 ORDER BY w.warehouse_id, p.product_id;
226
Ln 212, Col 45
Output
Result #25
warehouse_name | product_name | quantity_on_hand | reorder_level | last_updated
Central Warehouse | Laptop | 20 | 30 | 11/01/2026
Central Warehouse | Wireless Mouse | 150 | 50 | 11/01/2026
Central Warehouse | Keyboard | 25 | 40 | 11/01/2026
Western Warehouse | Laptop | 15 | 25 | 11/02/2026
Western Warehouse | Monitor | 60 | 30 | 11/02/2026
Western Warehouse | Headphones | 10 | 20 | 11/02/2026
Southern Warehouse | Wireless Mouse | 35 | 40 | 11/03/2026
Southern Warehouse | Keyboard | 80 | 30 | 11/03/2026
OK, 8 records retrieved in 1.005ms
View:
Pages:
Edit:

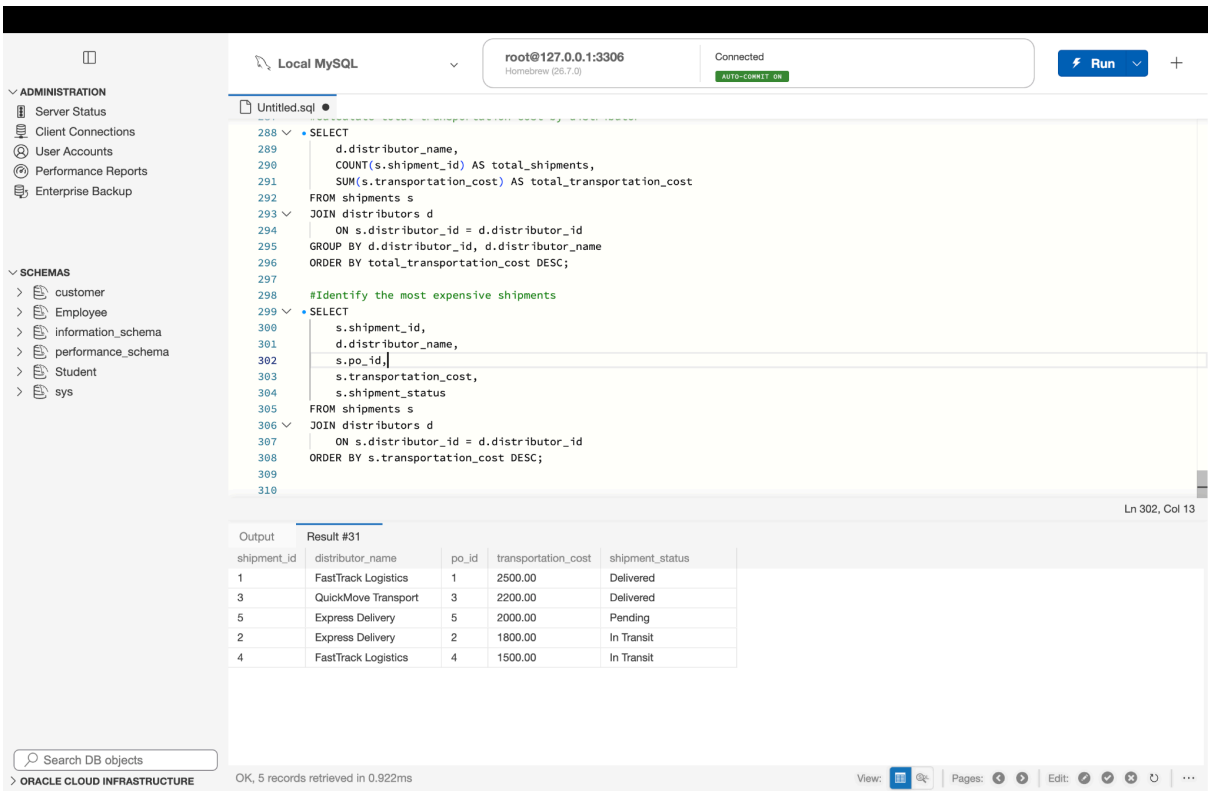

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
213
214 SELECT
215 w.warehouse_name,
216 p.product_name,
217 i.quantity_on_hand,
218 i.reorder_level,
219 i.last_updated
220 FROM inventory i
221 JOIN warehouses w
222 ON i.warehouse_id = w.warehouse_id
223 JOIN products p
224 ON i.product_id = p.product_id
225 ORDER BY w.warehouse_id, p.product_id;
226
227 SELECT
228 w.warehouse_name,
229 p.product_name,
230 i.quantity_on_hand,
231 i.reorder_level
232 FROM inventory i
233 JOIN warehouses w
234 ON i.warehouse_id = w.warehouse_id
235 JOIN products p
236 ON i.product_id = p.product_id
237 WHERE i.quantity_on_hand < i.reorder_level
238 ORDER BY i.quantity_on_hand;
239
240
Ln 239, Col 1
Output
Result #26
warehouse_name | product_name | quantity_on_hand | reorder_level
Western Warehouse | Headphones | 10 | 20
Western Warehouse | Laptop | 15 | 25
Central Warehouse | Laptop | 20 | 30
Central Warehouse | Keyboard | 25 | 40
Southern Warehouse | Wireless Mouse | 35 | 40
OK, 5 records retrieved in 0.639ms
View:
Pages:
Edit:

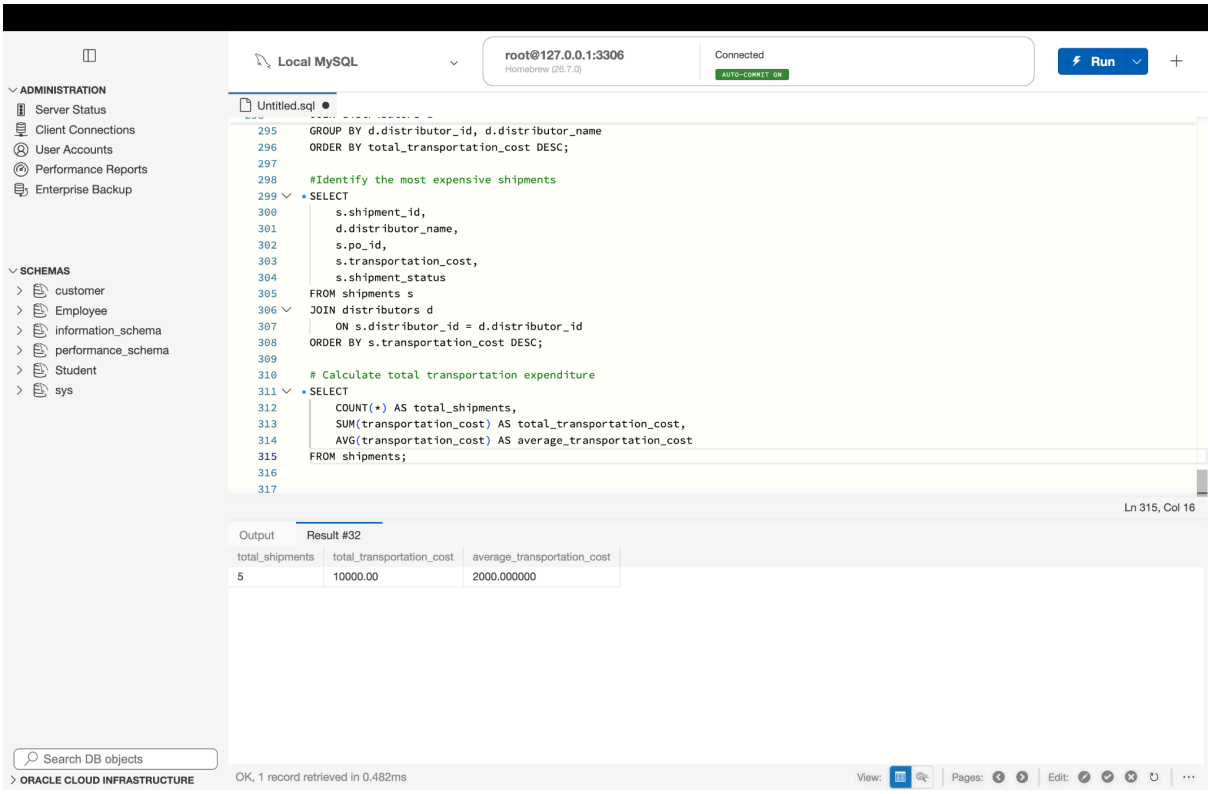

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
227 SELECT
232 FROM inventory i
233 JOIN warehouses w
234 ON i.warehouse_id = w.warehouse_id
235 JOIN products p
236 ON i.product_id = p.product_id
237 WHERE i.quantity_on_hand < i.reorder_level
238 ORDER BY i.quantity_on_hand;
239
240 SELECT
241 po.po_id,
242 s.supplier_name,
243 p.product_name,
244 po.order_status,
245 poi.quantity_ordered,
246 poi.quantity_received,
247 (poi.quantity_ordered - poi.quantity_received) AS pending_quantity
248 FROM purchase_orders po
249 JOIN suppliers s
250 ON po.supplier_id = s.supplier_id
251 JOIN purchase_order_items poi
252 ON po.po_id = poi.po_id
253 JOIN products p
254 ON poi.product_id = p.product_id
255 WHERE po.order_status IN ('Pending', 'In Transit')
256 AND poi.quantity_ordered > poi.quantity_received
257 ORDER BY po.po_id;
258
Ln 258, Col 1
Output
Result #27
po_id supplier_name product_name order_status quantity_ordered quantity_received pending_quantity
2 Global Supplies Keyboard Pending 80 0 80
4 Reliable Traders Headphones In Transit 60 30 30
5 ABC Components Wireless Mouse Pending 100 0 100
5 ABC Components Keyboard Pending 50 0 50
OK, 4 records retrieved in 0.759ms
View:
Pages:
Edit:

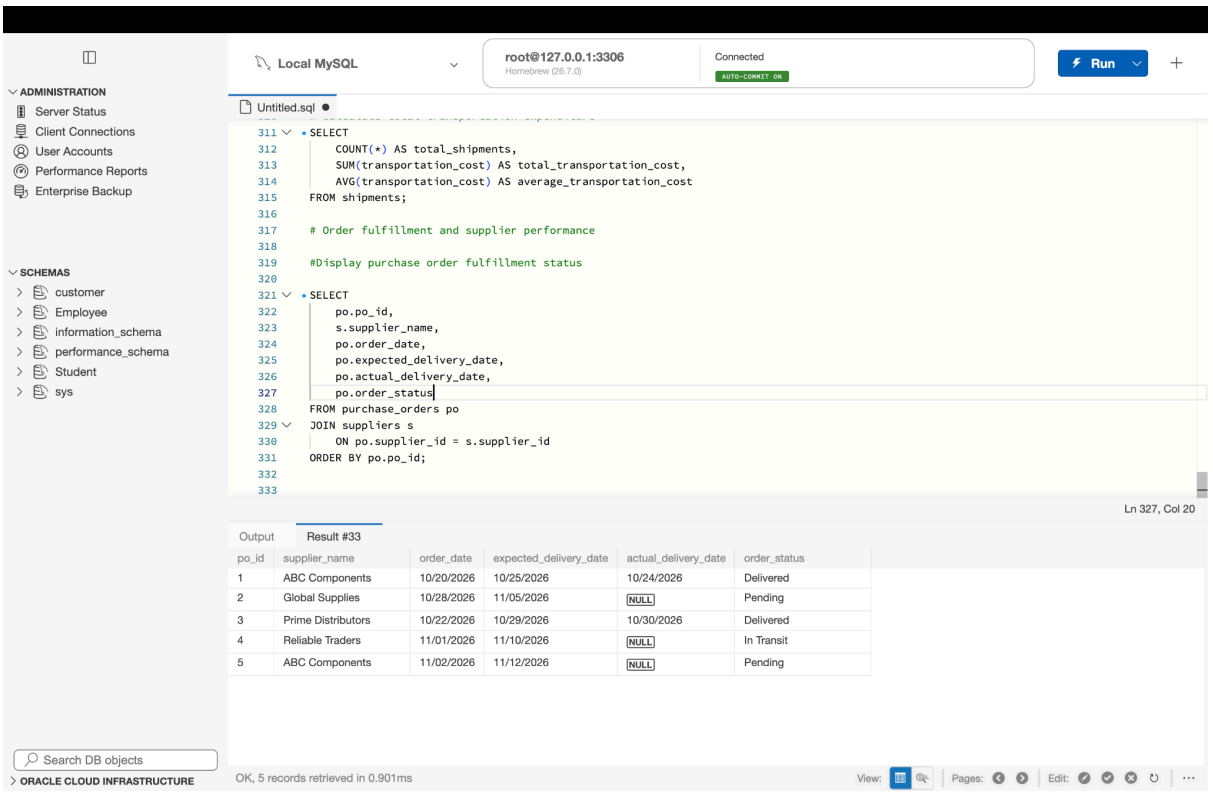

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
240 SELECT
243 p.product_name,
244 po.order_status,
245 poi.quantity_ordered,
246 poi.quantity_received,
247 (poi.quantity_ordered - poi.quantity_received) AS pending_quantity
248 FROM purchase_orders po
249 JOIN suppliers s
250 ON po.supplier_id = s.supplier_id
251 JOIN purchase_order_items poi
252 ON po.po_id = poi.po_id
253 JOIN products p
254 ON poi.product_id = p.product_id
255 WHERE po.order_status IN ('Pending', 'In Transit')
256 AND poi.quantity_ordered > poi.quantity_received
257 ORDER BY po.po_id;
258
259 SELECT
260 w.warehouse_name,
261 SUM(i.quantity_on_hand) AS total_units,
262 SUM(i.quantity_on_hand * p.unit_price) AS total_inventory_value
263 FROM inventory i
264 JOIN warehouses w
265 ON i.warehouse_id = w.warehouse_id
266 JOIN products p
267 ON i.product_id = p.product_id
268 GROUP BY w.warehouse_id, w.warehouse_name
269 ORDER BY total_inventory_value DESC;
Ln 269, Col 37
Output
Result #28
warehouse_name total_units total_inventory_value
Western Warehouse 85 1570000.00
Central Warehouse 195 1257500.00
Southern Warehouse 115 148000.00
OK, 3 records retrieved in 1.382ms
View:
Pages:
Edit:

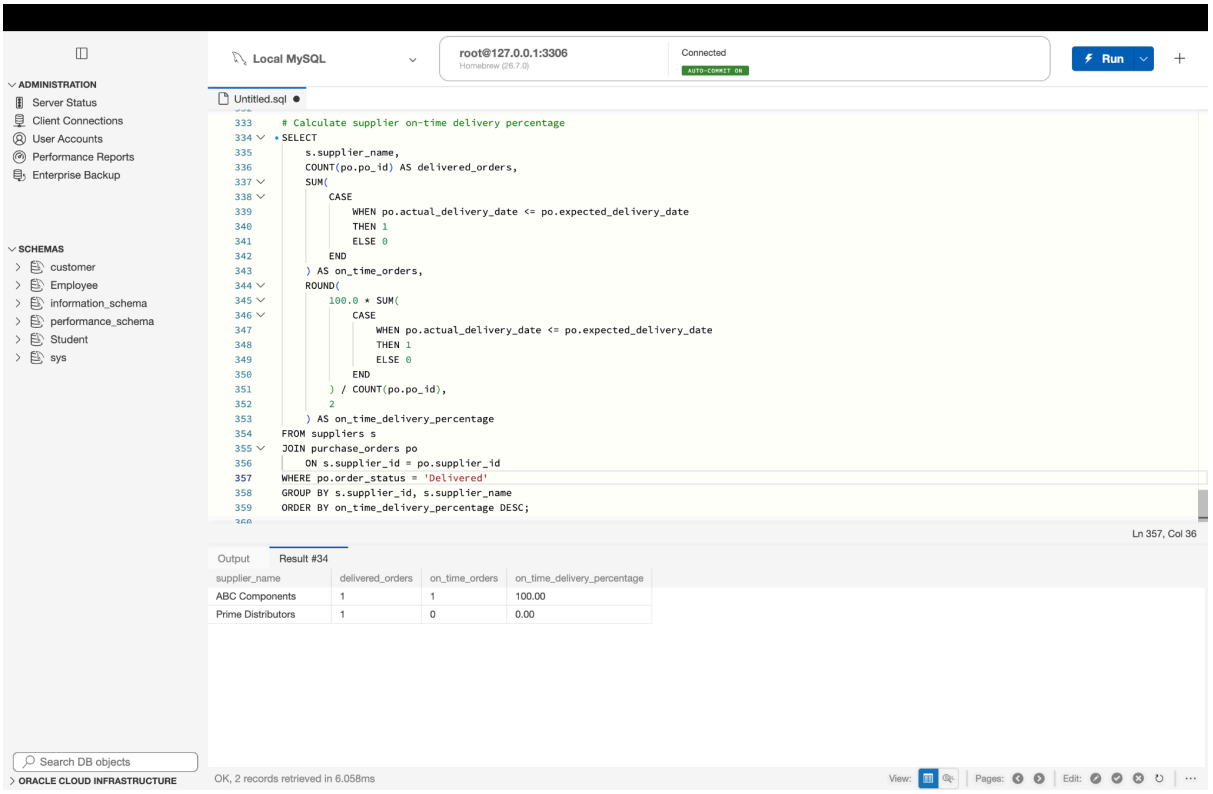

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
255 WHERE po.order_status IN ('Pending', 'In Transit')
257 ORDER BY po.po_id;
259 SELECT
260 w.warehouse_name,
261 SUM(i.quantity_on_hand) AS total_units,
262 SUM(i.quantity_on_hand * p.unit_price) AS total_inventory_value
263 FROM inventory i
264 JOIN warehouses w
265 ON i.warehouse_id = w.warehouse_id
266 JOIN products p
267 ON i.product_id = p.product_id
268 GROUP BY w.warehouse_id, w.warehouse_name
269 ORDER BY total_inventory_value DESC;
271 SELECT
272 w.warehouse_name,
273 p.product_name,
274 i.quantity_on_hand,
275 i.reorder_level,
276 GREATEST(i.reorder_level - i.quantity_on_hand, 0)
277 AS recommended_replenishment
278 FROM inventory i
279 JOIN warehouses w
280 ON i.warehouse_id = w.warehouse_id
281 JOIN products p
282 ON i.product_id = p.product_id
283 ORDER BY w.warehouse_id, p.product_id;
Ln 283, Col 39
Output
Result #29
warehouse_name | product_name | quantity_on_hand | reorder_level | recommended_replenishment
Central Warehouse | Laptop | 20 | 30 | 10
Central Warehouse | Wireless Mouse | 150 | 50 | 0
Central Warehouse | Keyboard | 25 | 40 | 15
Western Warehouse | Laptop | 15 | 25 | 10
Western Warehouse | Monitor | 60 | 30 | 0
Western Warehouse | Headphones | 10 | 20 | 10
Southern Warehouse | Wireless Mouse | 35 | 40 | 5
Southern Warehouse | Keyboard | 80 | 30 | 0
OK, 8 records retrieved in 2.66ms
View:
Pages:
Edit:

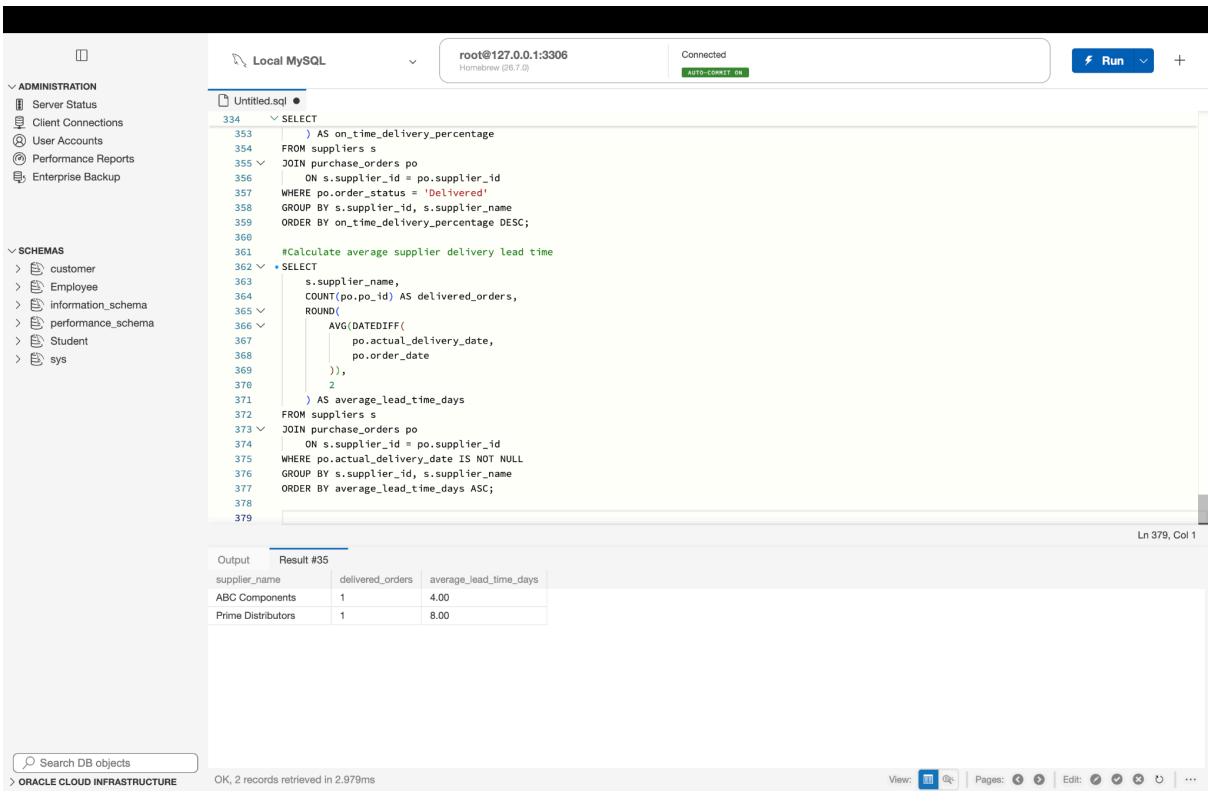

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
271 SELECT
276 GREATEST(i.reorder_level - i.quantity_on_hand, 0)
278 FROM inventory i
279 JOIN warehouses w
280 ON i.warehouse_id = w.warehouse_id
281 JOIN products p
282 ON i.product_id = p.product_id
283 ORDER BY w.warehouse_id, p.product_id;
285 #reducing transportation costs and improving order fulfillment
287 #Calculate total transportation cost by distributor
288 SELECT
289 d.distributor_name,
290 COUNT(s.shipment_id) AS total_shipments,
291 SUM(s.transportation_cost) AS total_transportation_cost
292 FROM shipments s
293 JOIN distributors d
294 ON s.distributor_id = d.distributor_id
295 GROUP BY d.distributor_id, d.distributor_name
296 ORDER BY total_transportation_cost DESC;
Ln 295, Col 46
Output
Result #30
distributor_name | total_shipments | total_transportation_cost
FastTrack Logistics | 2 | 4000.00
Express Delivery | 2 | 3800.00
QuickMove Transport | 1 | 2200.00
OK, 3 records retrieved in 0.737ms
View:
Pages:
Edit:

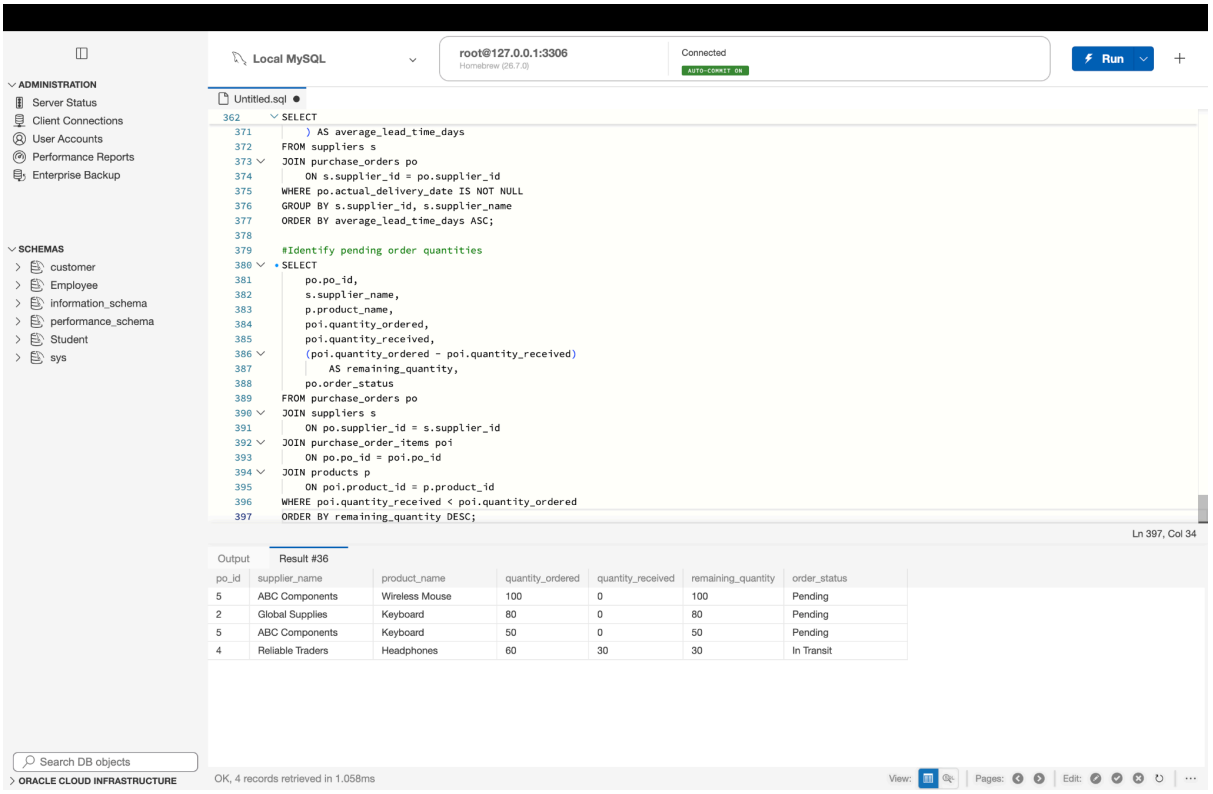

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
288 SELECT
289 d.distributor_name,
290 COUNT(s.shipment_id) AS total_shipments,
291 SUM(s.transportation_cost) AS total_transportation_cost
292 FROM shipments s
293 JOIN distributors d
294 ON s.distributor_id = d.distributor_id
295 GROUP BY d.distributor_id, d.distributor_name
296 ORDER BY total_transportation_cost DESC;
297
298 #Identify the most expensive shipments
299 SELECT
300 s.shipment_id,
301 d.distributor_name,
302 s.po_id,
303 s.transportation_cost,
304 s.shipment_status
305 FROM shipments s
306 JOIN distributors d
307 ON s.distributor_id = d.distributor_id
308 ORDER BY s.transportation_cost DESC;
309
310
Ln 302, Col 13
Output
Result #31
shipment_id distributor_name po_id transportation_cost shipment_status
1 FastTrack Logistics 1 2500.00 Delivered
3 QuickMove Transport 3 2200.00 Delivered
5 Express Delivery 5 2000.00 Pending
2 Express Delivery 2 1800.00 In Transit
4 FastTrack Logistics 4 1500.00 In Transit
OK, 5 records retrieved in 0.922ms
View:
Pages:
Edit:

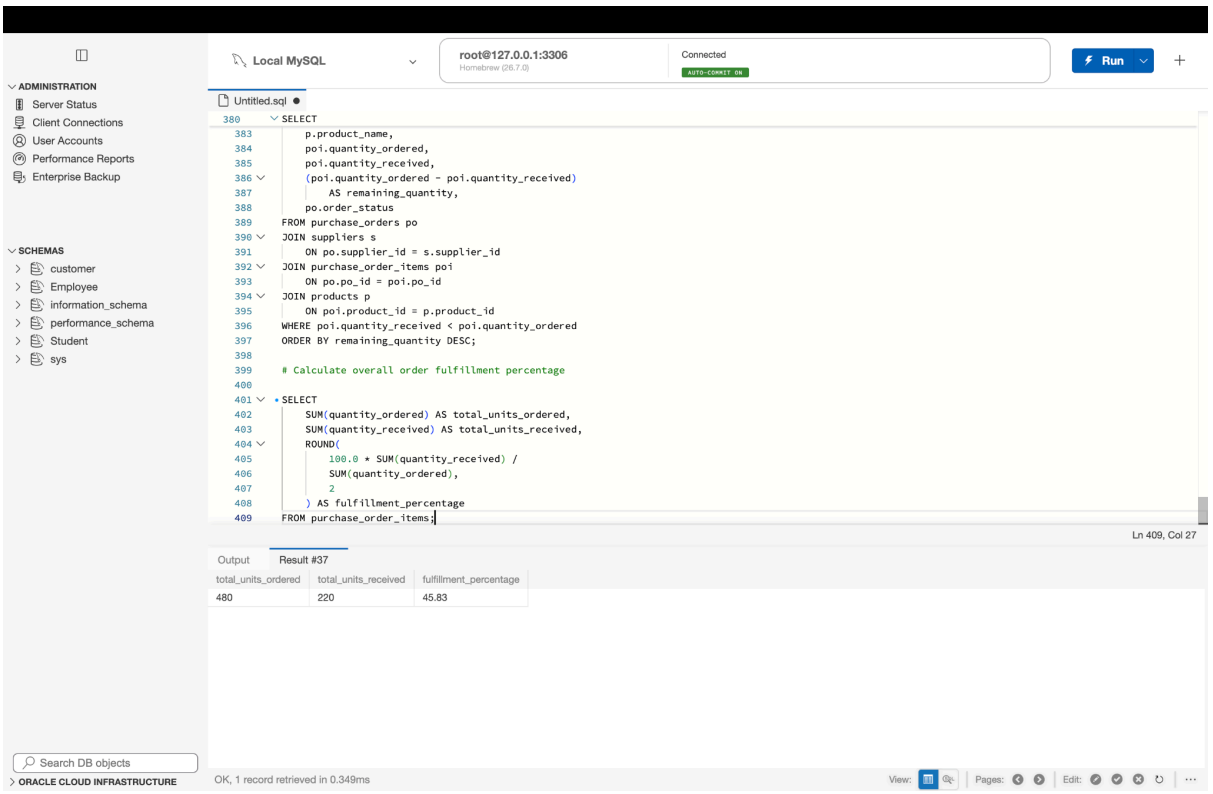

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
295 GROUP BY d.distributor_id, d.distributor_name
296 ORDER BY total_transportation_cost DESC;
297
298 #Identify the most expensive shipments
299 SELECT
300 s.shipment_id,
301 d.distributor_name,
302 s.po_id,
303 s.transportation_cost,
304 s.shipment_status
305 FROM shipments s
306 JOIN distributors d
307 ON s.distributor_id = d.distributor_id
308 ORDER BY s.transportation_cost DESC;
309
310 # Calculate total transportation expenditure
311 SELECT
312 COUNT(*) AS total_shipments,
313 SUM(transportation_cost) AS total_transportation_cost,
314 AVG(transportation_cost) AS average_transportation_cost
315 FROM shipments;
316
317
Ln 315, Col 16
Output
Result #32
total_shipments total_transportation_cost average_transportation_cost
5 10000.00 2000.000000
OK, 1 record retrieved in 0.482ms
View:
Pages:
Edit:

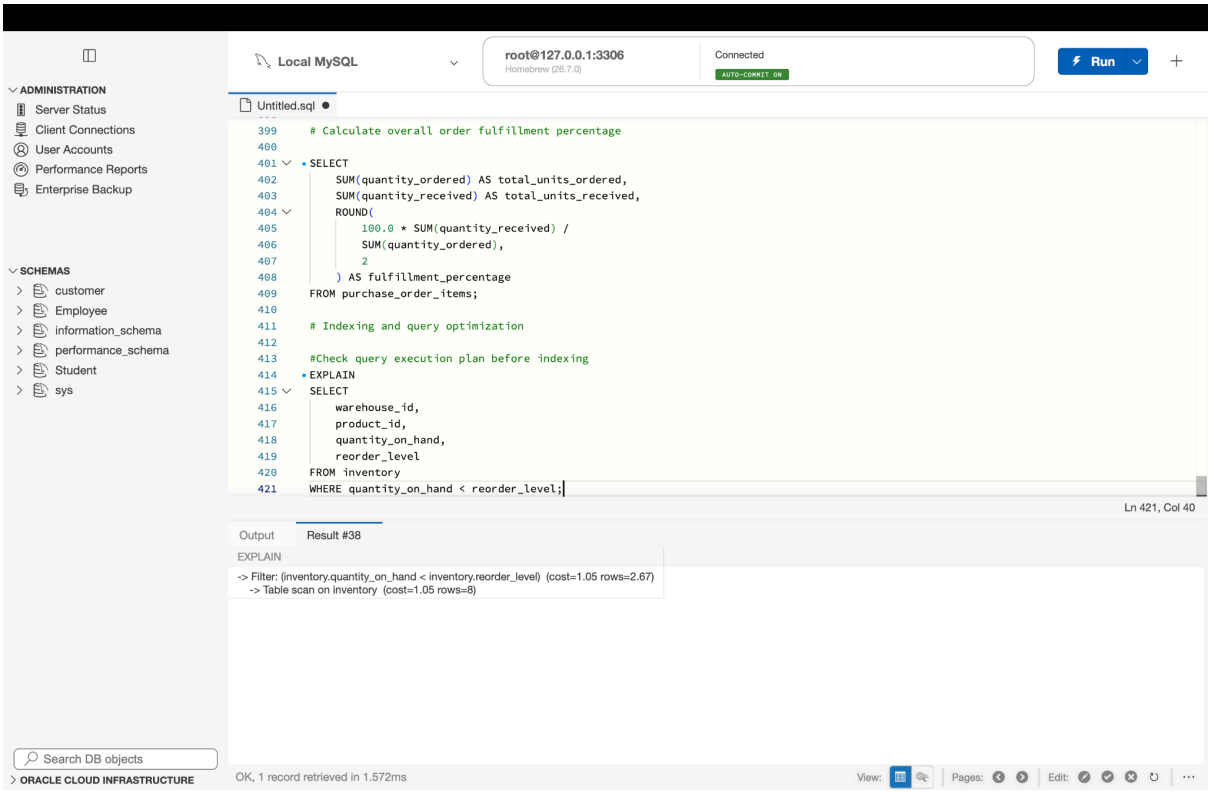

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
311 SELECT
312 COUNT(*) AS total_shipments,
313 SUM(transportation_cost) AS total_transportation_cost,
314 AVG(transportation_cost) AS average_transportation_cost
315 FROM shipments;
316
317 # Order fulfillment and supplier performance
318
319 #Display purchase order fulfillment status
320
321 SELECT
322 po.po_id,
323 s.supplier_name,
324 po.order_date,
325 po.expected_delivery_date,
326 po.actual_delivery_date,
327 po.order_status
328 FROM purchase_orders po
329 JOIN suppliers s
330 ON po.supplier_id = s.supplier_id
331 ORDER BY po.po_id;
332
333
Ln 327, Col 20
Output
Result #33
po_id | supplier_name | order_date | expected_delivery_date | actual_delivery_date | order_status
1 | ABC Components | 10/20/2026 | 10/25/2026 | 10/24/2026 | Delivered
2 | Global Supplies | 10/28/2026 | 11/05/2026 | NULL | Pending
3 | Prime Distributors | 10/22/2026 | 10/29/2026 | 10/30/2026 | Delivered
4 | Reliable Traders | 11/01/2026 | 11/10/2026 | NULL | In Transit
5 | ABC Components | 11/02/2026 | 11/12/2026 | NULL | Pending
OK, 5 records retrieved in 0.901ms
View:
Pages:
Edit:

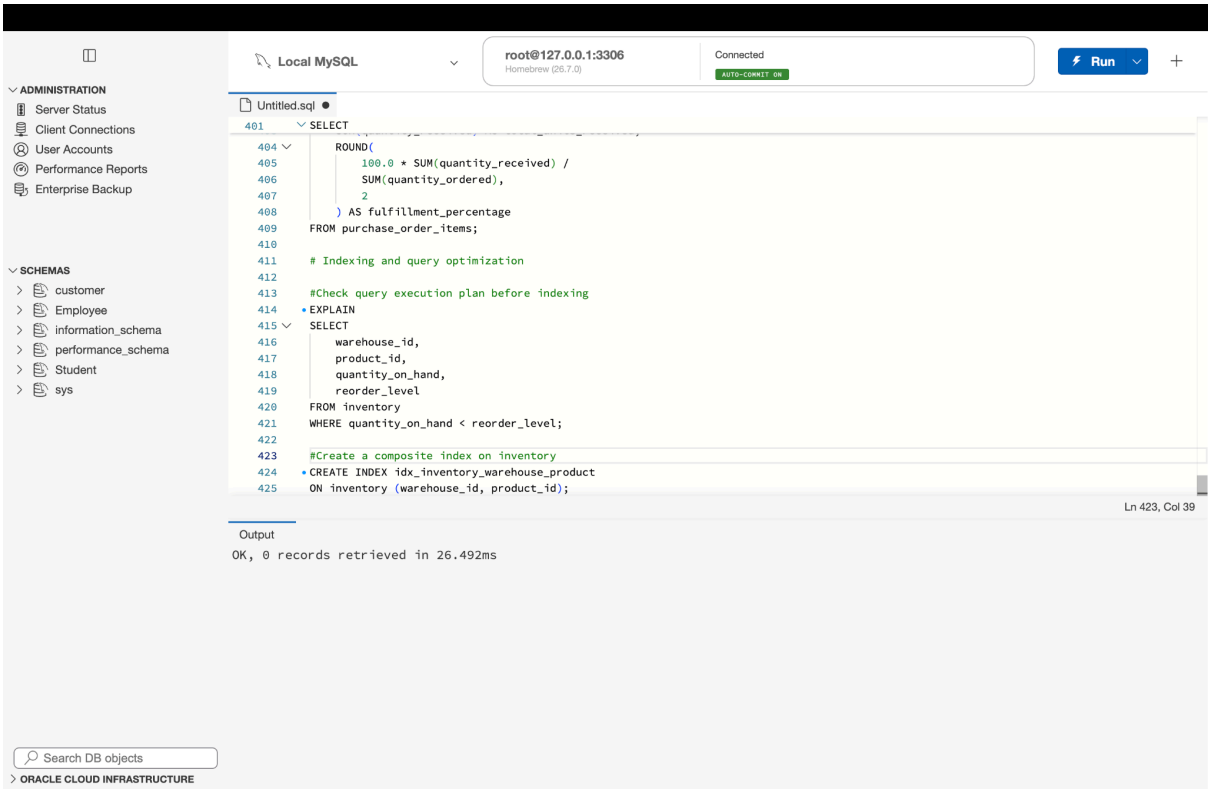

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
333 # Calculate supplier on-time delivery percentage
334 SELECT
335 s.supplier_name,
336 COUNT(po.po_id) AS delivered_orders,
337 SUM(
338 CASE
339 WHEN po.actual_delivery_date <= po.expected_delivery_date
340 THEN 1
341 ELSE 0
342 END
343) AS on_time_orders,
344 ROUND(
345 100.0 * SUM(
346 CASE
347 WHEN po.actual_delivery_date <= po.expected_delivery_date
348 THEN 1
349 ELSE 0
350 END
351) / COUNT(po.po_id),
352 2
353) AS on_time_delivery_percentage
354 FROM suppliers s
355 JOIN purchase_orders po
356 ON s.supplier_id = po.supplier_id
357 WHERE po.order_status = 'Delivered'
358 GROUP BY s.supplier_id, s.supplier_name
359 ORDER BY on_time_delivery_percentage DESC;
Ln 357, Col 36
Output
Result #34
supplier_name | delivered_orders | on_time_orders | on_time_delivery_percentage
ABC Components | 1 | 1 | 100.00
Prime Distributors | 1 | 0 | 0.00
OK, 2 records retrieved in 6.058ms
View:
Pages:
Edit:

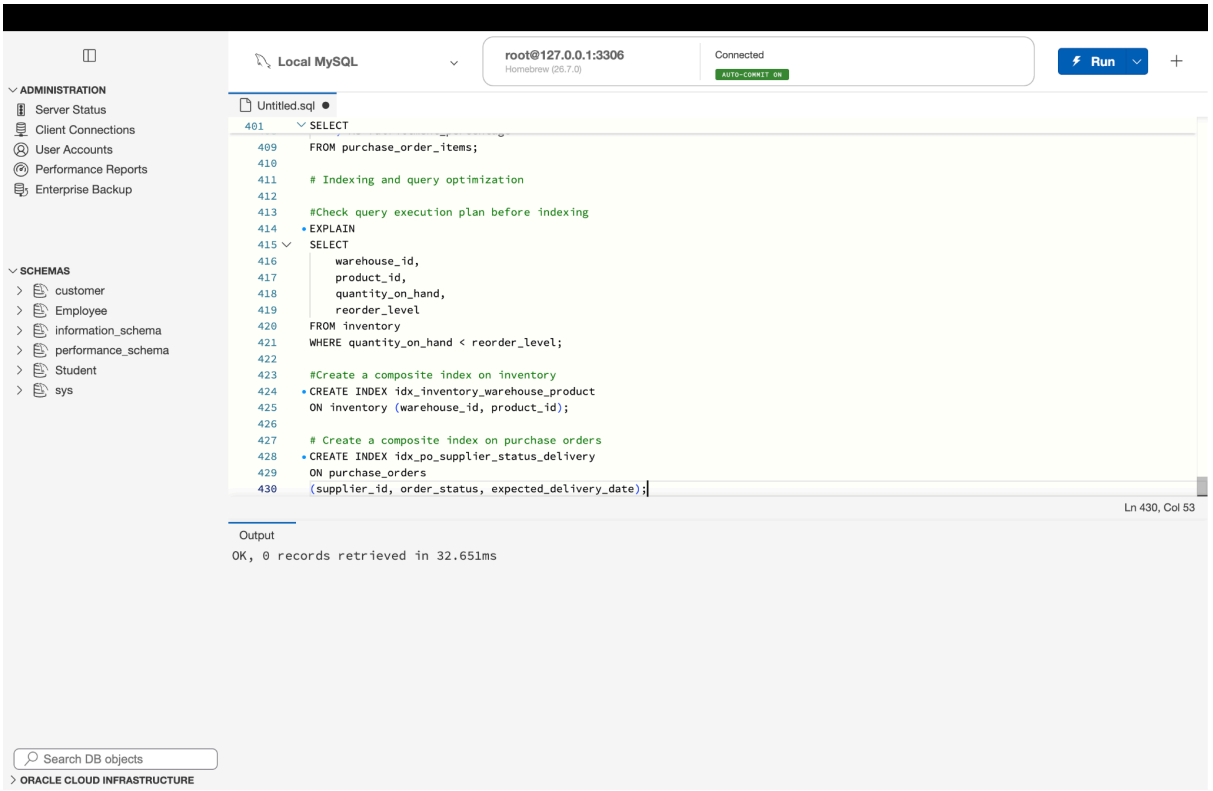

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
SELECT
) AS on_time_delivery_percentage
FROM suppliers s
JOIN purchase_orders po
ON s.supplier_id = po.supplier_id
WHERE po.order_status = 'Delivered'
GROUP BY s.supplier_id, s.supplier_name
ORDER BY on_time_delivery_percentage DESC;
#Calculate average supplier delivery lead time
SELECT
s.supplier_name,
COUNT(po.po_id) AS delivered_orders,
ROUND(
AVG(DATEDIFF(
po.actual_delivery_date,
po.order_date
)),
2
) AS average_lead_time_days
FROM suppliers s
JOIN purchase_orders po
ON s.supplier_id = po.supplier_id
WHERE po.actual_delivery_date IS NOT NULL
GROUP BY s.supplier_id, s.supplier_name
ORDER BY average_lead_time_days ASC;
Ln 379, Col 1
Output
Result #35
supplier_name
delivered_orders
average_lead_time_days
ABC Components
1
4.00
Prime Distributors
1
8.00
OK, 2 records retrieved in 2.979ms
View:
Pages:
Edit:

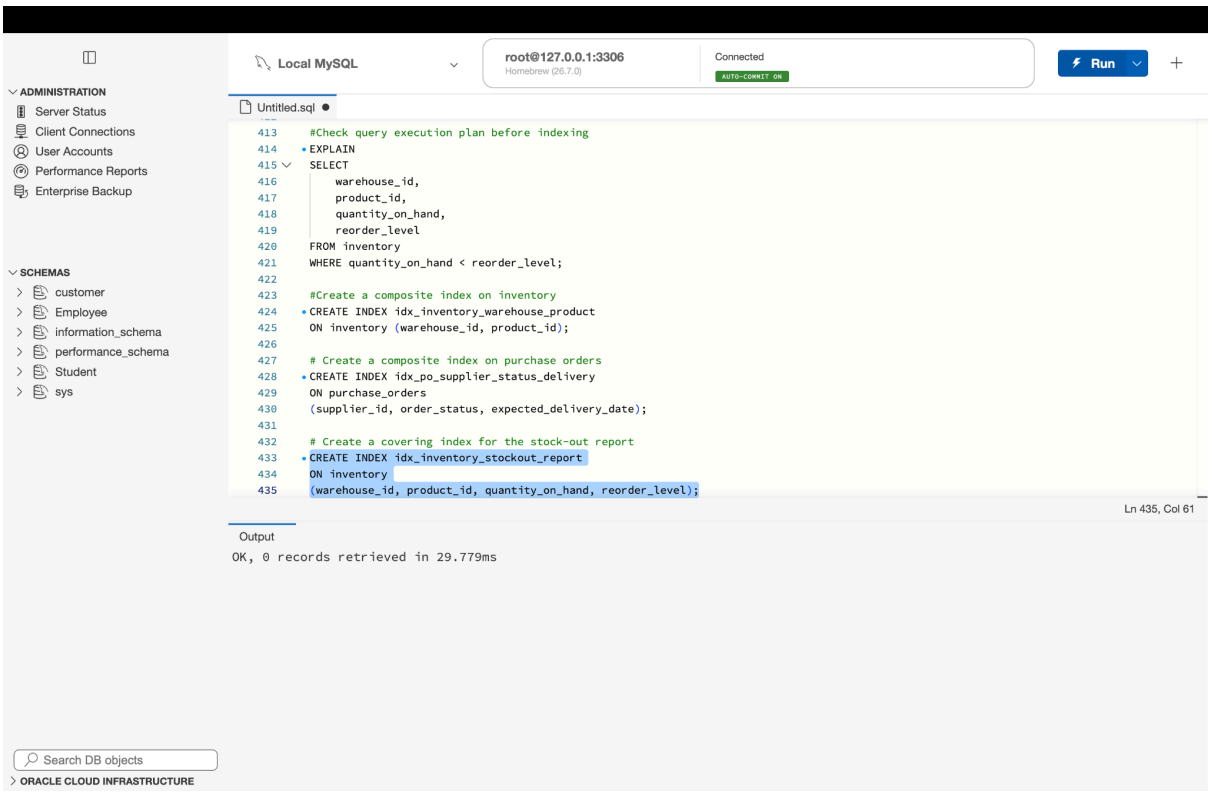

Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
SELECT
) AS average_lead_time_days
FROM suppliers s
JOIN purchase_orders po
ON s.supplier_id = po.supplier_id
WHERE po.actual_delivery_date IS NOT NULL
GROUP BY s.supplier_id, s.supplier_name
ORDER BY average_lead_time_days ASC;
#Identify pending order quantities
SELECT
po.po_id,
s.supplier_name,
p.product_name,
poi.quantity_ordered,
poi.quantity_received,
(poi.quantity_ordered - poi.quantity_received)
AS remaining_quantity,
po.order_status
FROM purchase_orders po
JOIN suppliers s
ON po.supplier_id = s.supplier_id
JOIN purchase_order_items poi
ON po.po_id = poi.po_id
JOIN products p
ON poi.product_id = p.product_id
WHERE poi.quantity_received < poi.quantity_ordered
ORDER BY remaining_quantity DESC;
Ln 397, Col 34
Output
Result #36
po_id
supplier_name
product_name
quantity_ordered
quantity_received
remaining_quantity
order_status
5
ABC Components
Wireless Mouse
100
0
100
Pending
2
Global Supplies
Keyboard
80
0
80
Pending
5
ABC Components
Keyboard
50
0
50
Pending
4
Reliable Traders
Headphones
60
30
30
In Transit
OK, 4 records retrieved in 1.058ms
View:
Pages:
Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Untitled.sql
SELECT
p.product_name,
poi.quantity_ordered,
poi.quantity_received,
(poi.quantity_ordered - poi.quantity_received)
AS remaining_quantity,
po.order_status
FROM purchase_orders po
JOIN suppliers s
ON po.supplier_id = s.supplier_id
JOIN purchase_order_items poi
ON po.po_id = poi.po_id
JOIN products p
ON poi.product_id = p.product_id
WHERE poi.quantity_received < poi.quantity_ordered
ORDER BY remaining_quantity DESC;
Calculate overall order fulfillment percentage
SELECT
SUM(quantity_ordered) AS total_units_ordered,
SUM(quantity_received) AS total_units_received,
ROUND(
100.0 * SUM(quantity_received) /
SUM(quantity_ordered),
2
) AS fulfillment_percentage
FROM purchase_order_items;
Ln 409, Col 27
Output
Result #37
total_units_ordered
total_units_received
fulfillment_percentage
480
220
45.83
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
OK, 1 record retrieved in 0.349ms
View:
Pages:
Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Untitled.sql
Calculate overall order fulfillment percentage
SELECT
SUM(quantity_ordered) AS total_units_ordered,
SUM(quantity_received) AS total_units_received,
ROUND(
100.0 * SUM(quantity_received) /
SUM(quantity_ordered),
2
) AS fulfillment_percentage
FROM purchase_order_items;
Indexing and query optimization
#Check query execution plan before indexing
EXPLAIN
SELECT
warehouse_id,
product_id,
quantity_on_hand,
reorder_level
FROM inventory
WHERE quantity_on_hand < reorder_level;
Ln 421, Col 40
Output
Result #38
EXPLAIN
-> Filter: (inventory.quantity_on_hand < inventory.reorder_level) (cost=1.05 rows=2.67)
-> Table scan on inventory (cost=1.05 rows=8)
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
OK, 1 record retrieved in 1.572ms
View:
Pages:
Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
401 SELECT
404 ROUND(
405 100.0 * SUM(quantity_received) /
406 SUM(quantity_ordered),
407 2
408) AS fulfillment_percentage
409 FROM purchase_order_items;
410
411 # Indexing and query optimization
412
413 #Check query execution plan before indexing
414 EXPLAIN
415 SELECT
416 warehouse_id,
417 product_id,
418 quantity_on_hand,
419 reorder_level
420 FROM inventory
421 WHERE quantity_on_hand < reorder_level;
422
423 #Create a composite index on inventory
424 CREATE INDEX idx_inventory_warehouse_product
425 ON inventory (warehouse_id, product_id);
Ln 423, Col 39
Output
OK, 0 records retrieved in 26.492ms


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
401 SELECT
409 FROM purchase_order_items;
410
411 # Indexing and query optimization
412
413 #Check query execution plan before indexing
414 EXPLAIN
415 SELECT
416 warehouse_id,
417 product_id,
418 quantity_on_hand,
419 reorder_level
420 FROM inventory
421 WHERE quantity_on_hand < reorder_level;
422
423 #Create a composite index on inventory
424 CREATE INDEX idx_inventory_warehouse_product
425 ON inventory (warehouse_id, product_id);
426
427 # Create a composite index on purchase orders
428 CREATE INDEX idx_po_supplier_status_delivery
429 ON purchase_orders
430 (supplier_id, order_status, expected_delivery_date);
Ln 430, Col 53
Output
OK, 0 records retrieved in 32.651ms



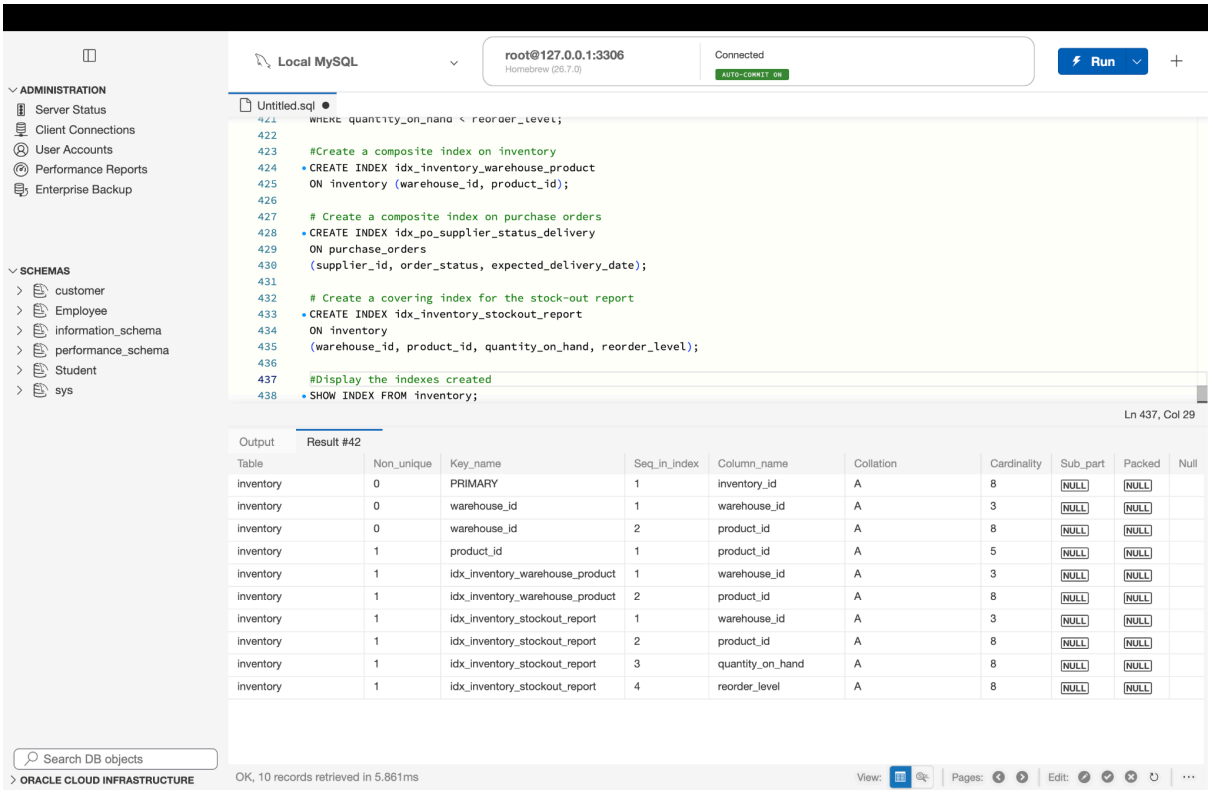
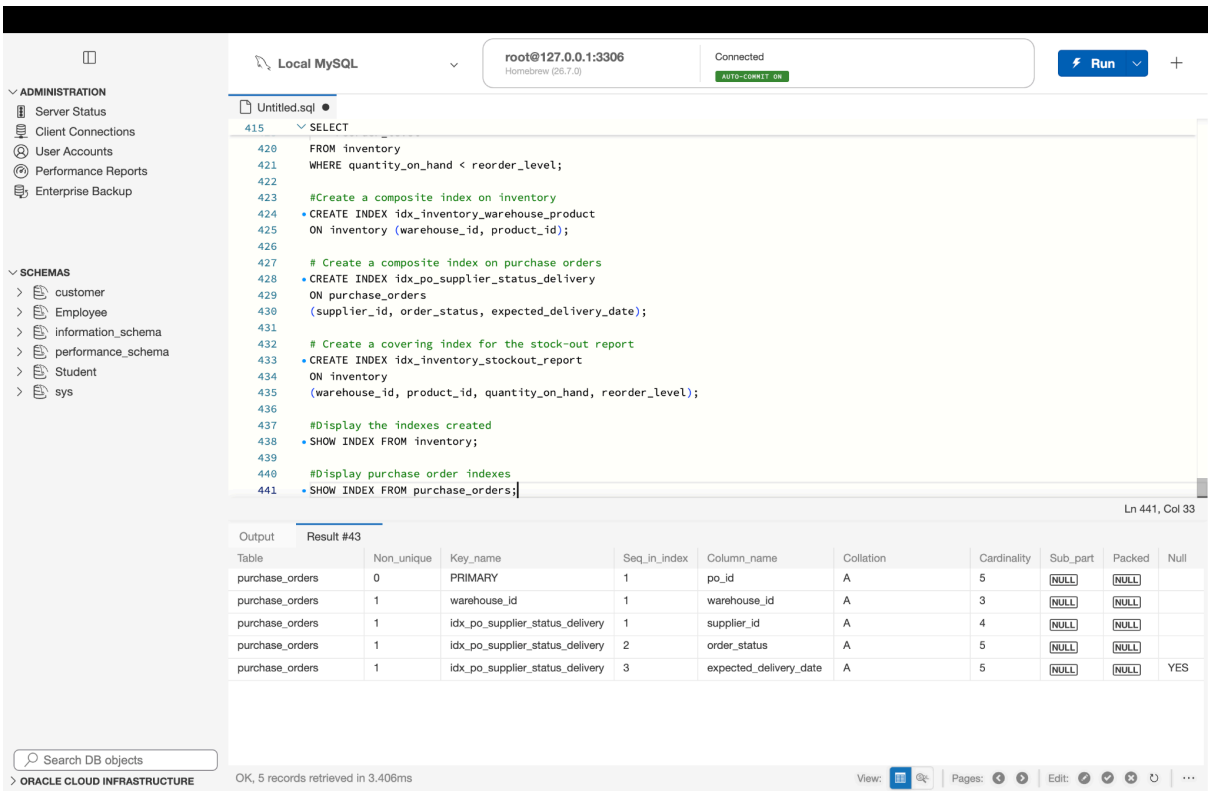
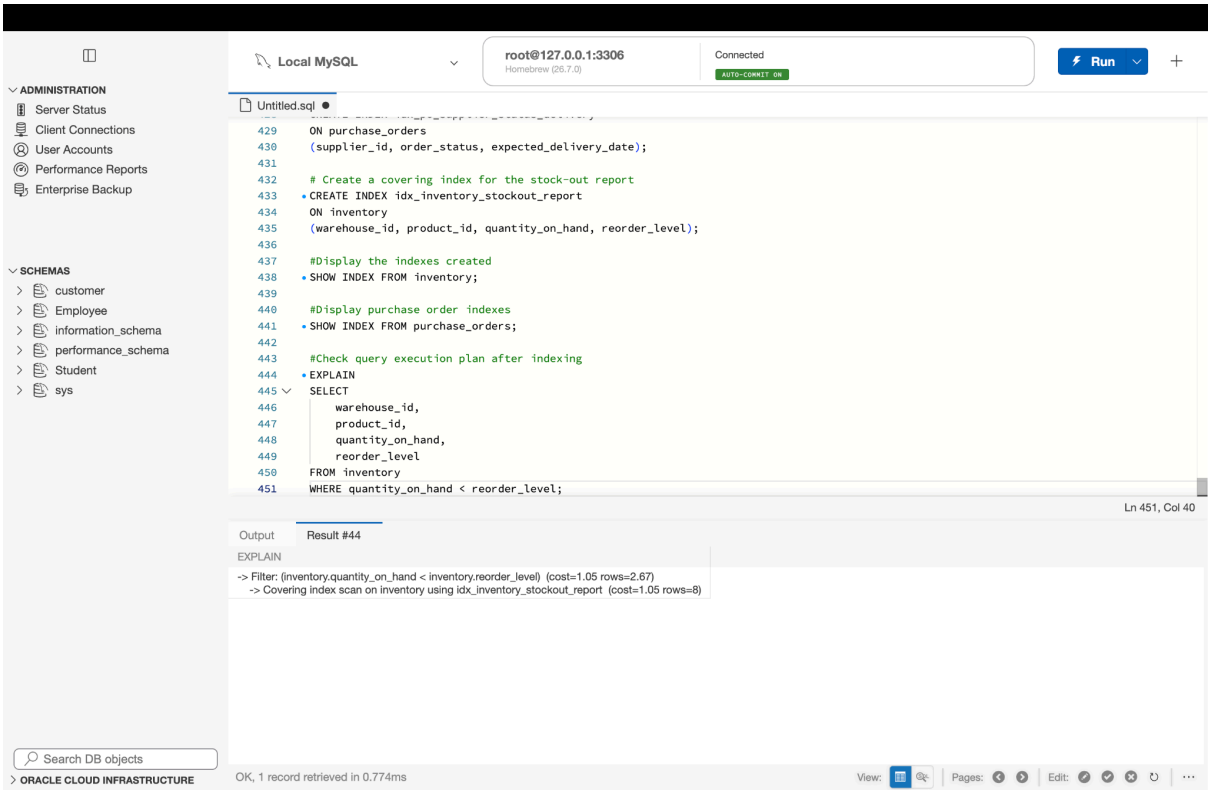


Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
inventory	0	PRIMARY	1	inventory_id	A	8	NULL	NULL	
inventory	0	warehouse_id	1	warehouse_id	A	3	NULL	NULL	
inventory	0	warehouse_id	2	product_id	A	8	NULL	NULL	
inventory	1	product_id	1	product_id	A	5	NULL	NULL	
inventory	1	idx_inventory_warehouse_product	1	warehouse_id	A	3	NULL	NULL	
inventory	1	idx_inventory_warehouse_product	2	product_id	A	8	NULL	NULL	
inventory	1	idx_inventory_stockout_report	1	warehouse_id	A	3	NULL	NULL	
inventory	1	idx_inventory_stockout_report	2	product_id	A	8	NULL	NULL	
inventory	1	idx_inventory_stockout_report	3	quantity_on_hand	A	8	NULL	NULL	
inventory	1	idx_inventory_stockout_report	4	reorder_level	A	8	NULL	NULL	


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
415 SELECT
420 FROM inventory
421 WHERE quantity_on_hand < reorder_level;
423 #Create a composite index on inventory
424 CREATE INDEX idx_inventory_warehouse_product
425 ON inventory (warehouse_id, product_id);
427 # Create a composite index on purchase orders
428 CREATE INDEX idx_po_supplier_status_delivery
429 ON purchase_orders
430 (supplier_id, order_status, expected_delivery_date);
432 # Create a covering index for the stock-out report
433 CREATE INDEX idx_inventory_stockout_report
434 ON inventory
435 (warehouse_id, product_id, quantity_on_hand, reorder_level);
437 #Display the indexes created
438 SHOW INDEX FROM inventory;
440 #Display purchase order indexes
441 SHOW INDEX FROM purchase_orders;
Ln 441, Col 33
Output
Result #43
Table Non_unique Key_name Seq_in_index Column_name Collation Cardinality Sub_part Packed Null
purchase_orders 0 PRIMARY 1 po_id A 5 NULL NULL
purchase_orders 1 warehouse_id 1 warehouse_id A 3 NULL NULL
purchase_orders 1 idx_po_supplier_status_delivery 1 supplier_id A 4 NULL NULL
purchase_orders 1 idx_po_supplier_status_delivery 2 order_status A 5 NULL NULL
purchase_orders 1 idx_po_supplier_status_delivery 3 expected_delivery_date A 5 NULL NULL YES
OK, 5 records retrieved in 3.406ms
View:
Pages:
Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
customer
Employee
information_schema
performance_schema
Student
sys
Search DB objects
ORACLE CLOUD INFRASTRUCTURE
Untitled.sql
429 ON purchase_orders
430 (supplier_id, order_status, expected_delivery_date);
432 # Create a covering index for the stock-out report
433 CREATE INDEX idx_inventory_stockout_report
434 ON inventory
435 (warehouse_id, product_id, quantity_on_hand, reorder_level);
437 #Display the indexes created
438 SHOW INDEX FROM inventory;
440 #Display purchase order indexes
441 SHOW INDEX FROM purchase_orders;
443 #Check query execution plan after indexing
444 EXPLAIN
445 SELECT
446 warehouse_id,
447 product_id,
448 quantity_on_hand,
449 reorder_level
450 FROM inventory
451 WHERE quantity_on_hand < reorder_level;
Ln 451, Col 40
Output
Result #44
EXPLAIN
-> Filter: (inventory.quantity_on_hand < inventory.reorder_level) (cost=1.05 rows=2.67)
-> Covering index scan on inventory using idx_inventory_stockout_report (cost=1.05 rows=8)
OK, 1 record retrieved in 0.774ms
View:
Pages:
Edit:

Local MySQL

root@127.0.0.1:3306
Homebrew (26.7.0)

Connected
AUTO-COMMIT ON

Run

Untitled.sql

```
452
453     #Create and use views
454
455     # Create a low-stock alert view
456
457   • CREATE OR REPLACE VIEW vw_low_stock_alert AS
458   ⌄ SELECT
459         w.warehouse_name,
460         p.product_name,
461         i.quantity_on_hand,
462         i.reorder_level,
463       ⌄ COALESCE(
464         ⌄     (
465         ⌄         SELECT SUM(
466               poi.quantity_ordered - poi.quantity_received
467           )
468           FROM purchase_orders po
469         ⌄ JOIN purchase_order_items poi
470               ON po.po_id = poi.po_id
471         ⌄ WHERE po.warehouse_id = i.warehouse_id
472               AND poi.product_id = i.product_id
473               AND po.order_status IN ('Pending', 'In Transit')
474               AND poi.quantity_ordered > poi.quantity_received
```

Ln 460, Col 20

Output

```
OK, 0 records retrieved in 5.555ms
```

Local MySQL

root@127.0.0.1:3306
Homebrew (26.7.0)

Connected
AUTO-COMMIT ON

Run

Untitled.sql

```
458   ⌄ SELECT
479   ⌄     COALESCE(
480   ⌄         (
485   ⌄             JOIN purchase_order_items poi
487   ⌄         WHERE po.warehouse_id = i.warehouse_id
488               AND poi.product_id = i.product_id
489               AND po.order_status IN ('Pending', 'In Transit')
490               AND poi.quantity_ordered > poi.quantity_received
491         ),
492         0
493     ) AS projected_stock
494   FROM inventory i
495   ⌄ JOIN warehouses w
496       ON i.warehouse_id = w.warehouse_id
497   ⌄ JOIN products p
498       ON i.product_id = p.product_id
499   WHERE i.quantity_on_hand < i.reorder_level;
500
501   # Display the low-stock alert report
502
503   • SELECT *
504   FROM vw_low_stock_alert
505   ORDER BY warehouse_name, product_name;
```

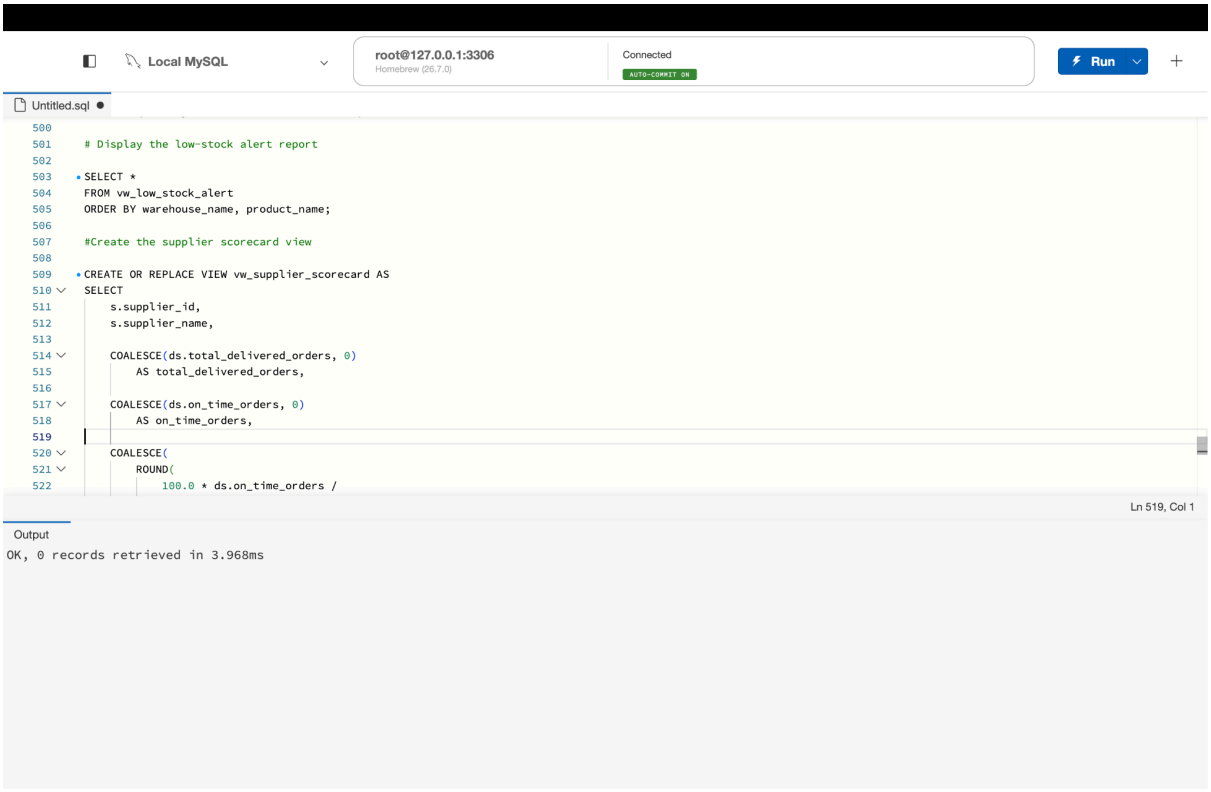
Ln 505, Col 39

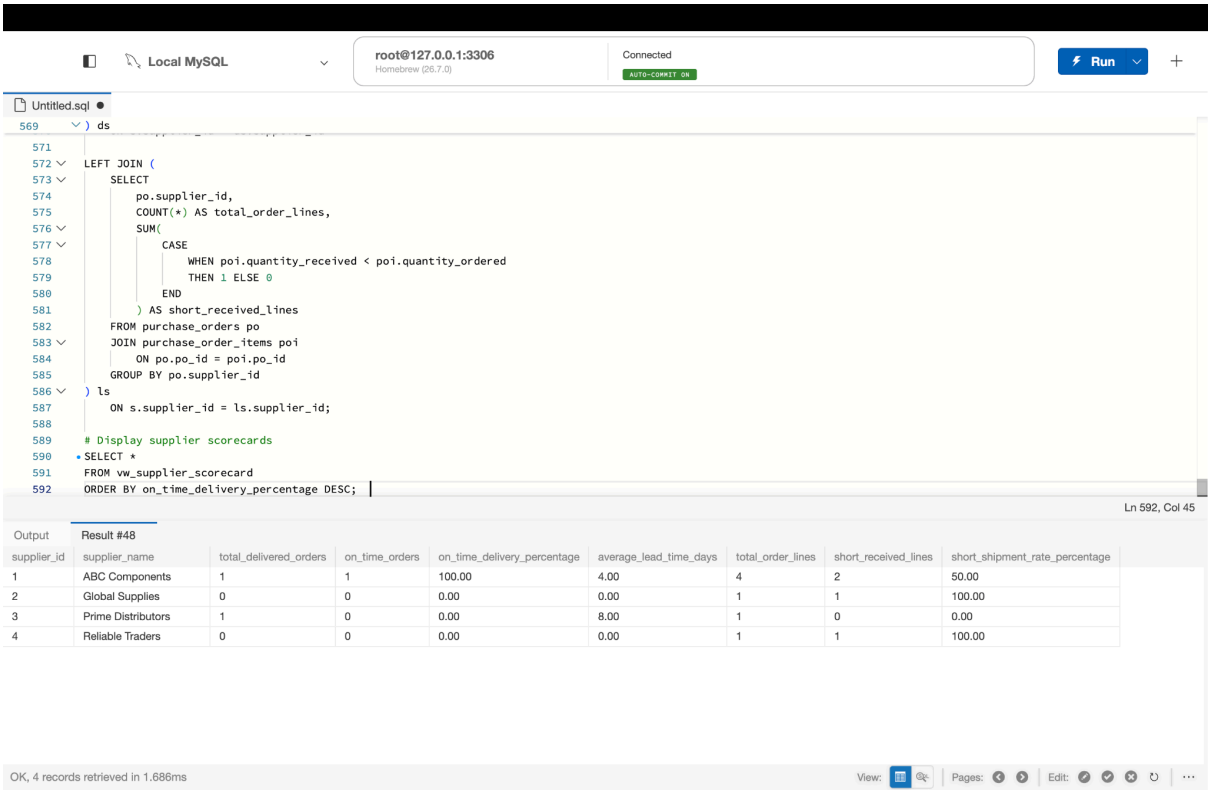
Output | Result #46

warehouse_name	product_name	quantity_on_hand	reorder_level	pending_po_quantity	projected_stock
Central Warehouse	Keyboard	25	40	80	105
Central Warehouse	Laptop	20	30	0	20
Southern Warehouse	Wireless Mouse	35	40	100	135
Western Warehouse	Headphones	10	20	30	40
Western Warehouse	Laptop	15	25	0	15

OK, 5 records retrieved in 1.756ms

View: Pages: Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
Untitled.sql
500
501 # Display the low-stock alert report
502
503 SELECT *
504 FROM vw_low_stock_alert
505 ORDER BY warehouse_name, product_name;
506
507 #Create the supplier scorecard view
508
509 CREATE OR REPLACE VIEW vw_supplier_scorecard AS
510 SELECT
511 s.supplier_id,
512 s.supplier_name,
513
514 COALESCE(ds.total_delivered_orders, 0)
515 AS total_delivered_orders,
516
517 COALESCE(ds.on_time_orders, 0)
518 AS on_time_orders,
519
520 COALESCE(
521 ROUND(
522 100.0 * ds.on_time_orders /
Ln 519, Col 1
Output
OK, 0 records retrieved in 3.968ms


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
Untitled.sql
569) ds
571
572 LEFT JOIN (
573 SELECT
574 po.supplier_id,
575 COUNT(*) AS total_order_lines,
576 SUM(
577 CASE
578 WHEN poi.quantity_received < poi.quantity_ordered
579 THEN 1 ELSE 0
580 END
581) AS short_received_lines
582 FROM purchase_orders po
583 JOIN purchase_order_items poi
584 ON po.po_id = poi.po_id
585 GROUP BY po.supplier_id
586) ls
587 ON s.supplier_id = ls.supplier_id;
588
589 # Display supplier scorecards
590 SELECT *
591 FROM vw_supplier_scorecard
592 ORDER BY on_time_delivery_percentage DESC;
Ln 592, Col 45
Output
Result #48
supplier_id | supplier_name | total_delivered_orders | on_time_orders | on_time_delivery_percentage | average_lead_time_days | total_order_lines | short_received_lines | short_shipment_rate_percentage
1 | ABC Components | 1 | 1 | 100.00 | 4.00 | 4 | 2 | 50.00
2 | Global Supplies | 0 | 0 | 0.00 | 0.00 | 1 | 1 | 100.00
3 | Prime Distributors | 1 | 0 | 0.00 | 8.00 | 1 | 0 | 0.00
4 | Reliable Traders | 0 | 0 | 0.00 | 0.00 | 1 | 1 | 100.00
OK, 4 records retrieved in 1.686ms
View:
Pages:
Edit:

Local MySQL

root@127.0.0.1:3306
Homebrew (26.7.0)

Connected
AUTO-COMMIT ON

Run

Untitled.sql

```
572 LEFT JOIN (
582     FROM purchase_orders po
583     JOIN purchase_order_items poi
584         ON po.po_id = poi.po_id
585     GROUP BY po.supplier_id
586 ) ls
587     ON s.supplier_id = ls.supplier_id;
588
589 # Display supplier scorecards
590 SELECT *
591 FROM vw_supplier_scorecard
592 ORDER BY on_time_delivery_percentage DESC;
593
594 # Check whether the views are updatable
595 SELECT
596     TABLE_NAME,
597     IS_UPDATABLE
598 FROM information_schema.VIEWS
599 WHERE TABLE_SCHEMA = DATABASE()
600   AND TABLE_NAME IN (
601         'vw_low_stock_alert',
602         'vw_supplier_scorecard'
603     );
```

Ln 603, Col 5

Output | Result #49

TABLE_NAME	IS_UPDATABLE
vw_low_stock_alert	YES
vw_supplier_scorecard	NO

OK, 2 records retrieved in 6.901ms

View: | Pages: | Edit:

Local MySQL

root@127.0.0.1:3306
Homebrew (26.7.0)

Connected
AUTO-COMMIT ON

Run

Untitled.sql

```
599 WHERE TABLE_SCHEMA = DATABASE()
604
605 # Rank primary and backup suppliers for each product
606 SELECT
607     product_name,
608     supplier_name,
609     average_lead_time_days,
610     average_unit_cost,
611     supplier_rank,
612     CASE
613         WHEN supplier_rank = 1 THEN 'Primary Supplier'
614         WHEN supplier_rank = 2 THEN 'Backup Supplier'
615         ELSE 'Other Supplier'
616     END AS supplier_category
617 FROM (
618     SELECT
619         product_id,
620         product_name,
621         supplier_name,
622         average_lead_time_days,
623         average_unit_cost,
624         ROW_NUMBER() OVER (
625             PARTITION BY product_id
```

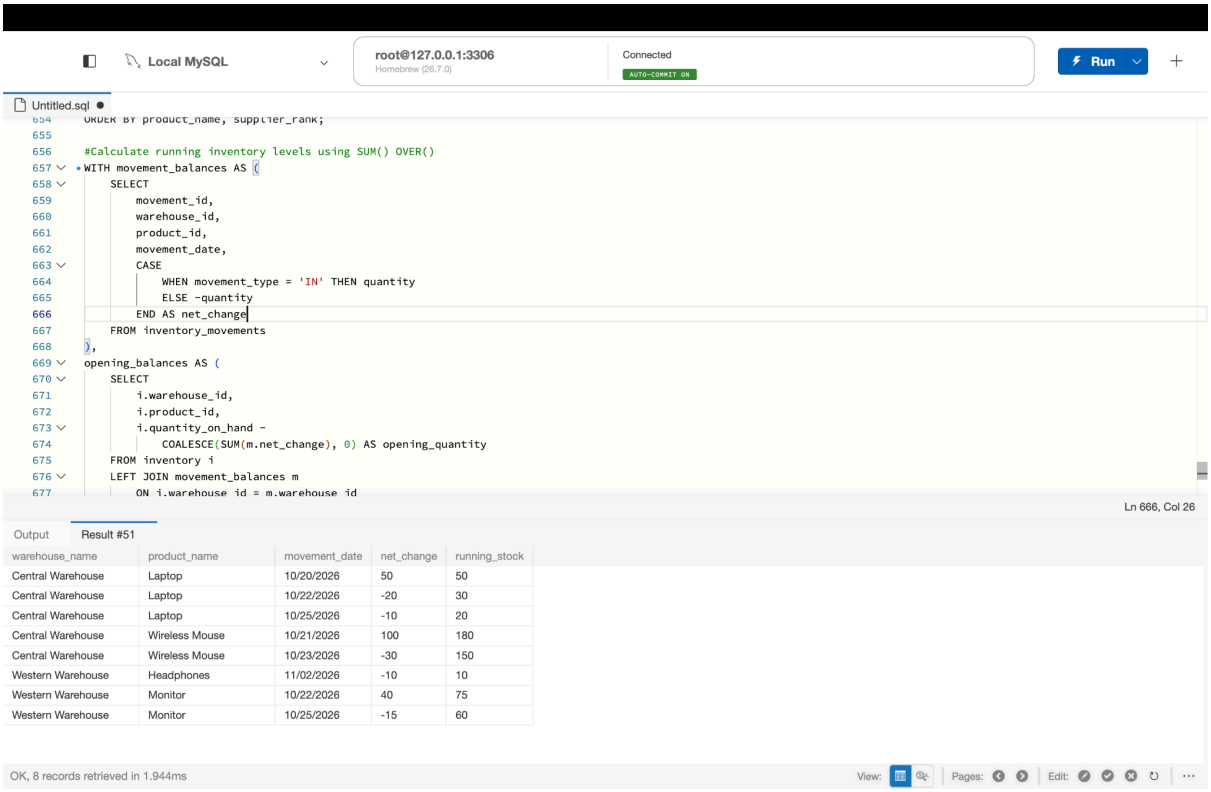
Ln 618, Col 11

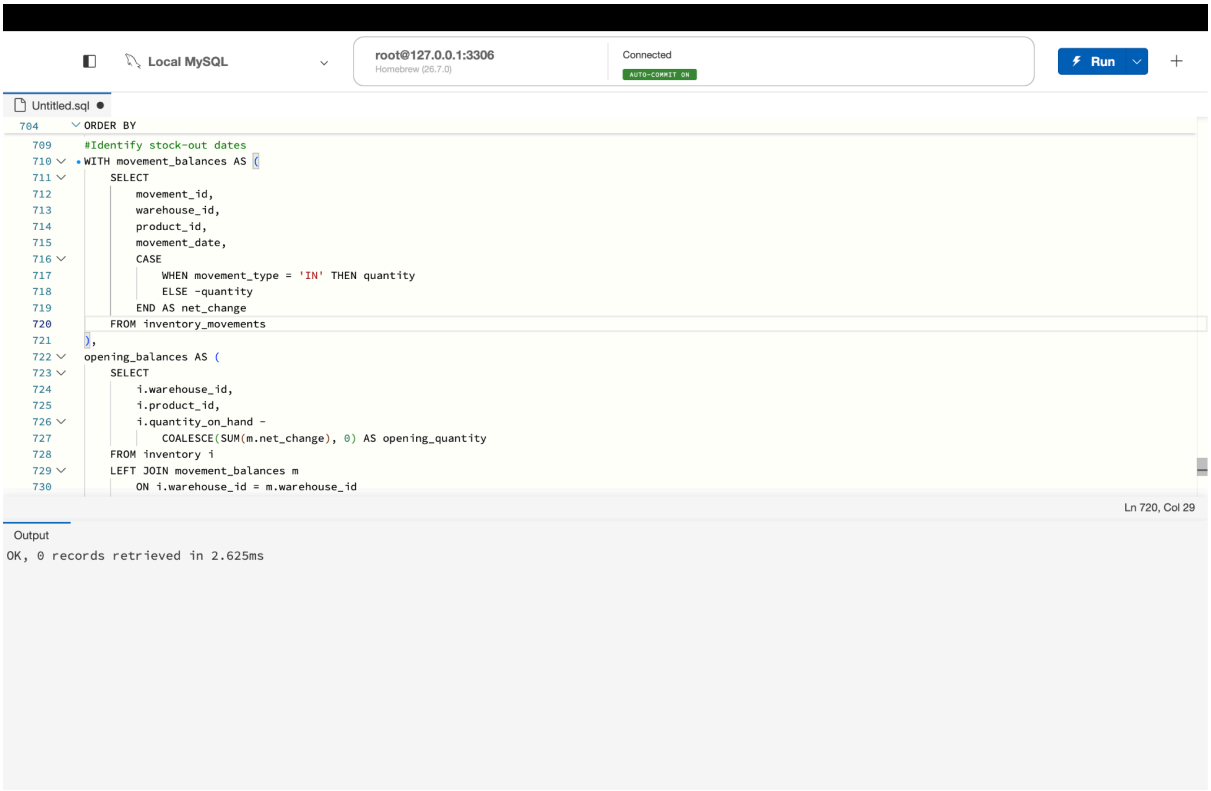
Output | Result #50

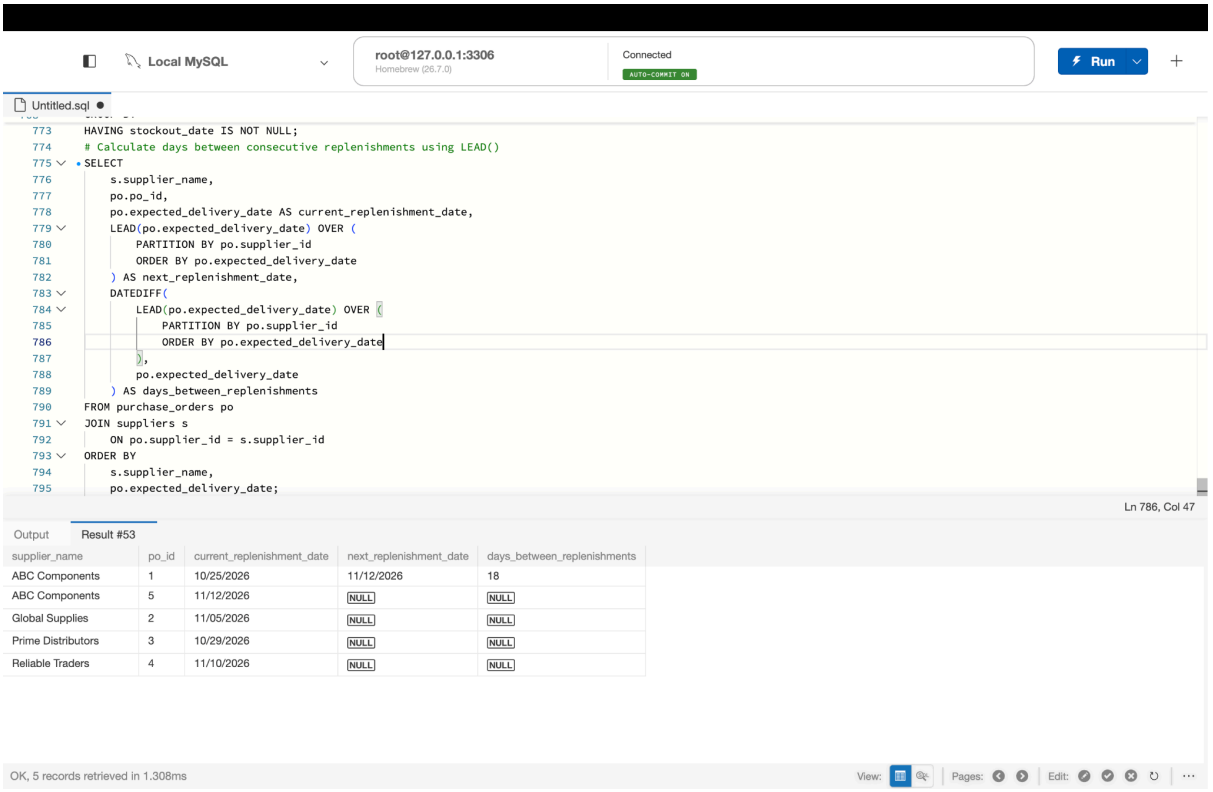
product_name	supplier_name	average_lead_time_days	average_unit_cost	supplier_rank	supplier_category
Laptop	ABC Components	4.0000	50000.000000	1	Primary Supplier
Monitor	Prime Distributors	8.0000	11000.000000	1	Primary Supplier
Wireless Mouse	ABC Components	4.0000	700.000000	1	Primary Supplier

OK, 3 records retrieved in 5.565ms

View: | Pages: | Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
Untitled.sql
ORDER BY product_name, supplier_rank;
#Calculate running inventory levels using SUM() OVER()
WITH movement_balances AS (
SELECT
movement_id,
warehouse_id,
product_id,
movement_date,
CASE
WHEN movement_type = 'IN' THEN quantity
ELSE -quantity
END AS net_change
FROM inventory_movements
),
opening_balances AS (
SELECT
i.warehouse_id,
i.product_id,
i.quantity_on_hand -
COALESCE(SUM(m.net_change), 0) AS opening_quantity
FROM inventory i
LEFT JOIN movement_balances m
ON i.warehouse_id = m.warehouse_id
Ln 666, Col 26
Output
Result #51
warehouse_name | product_name | movement_date | net_change | running_stock
Central Warehouse | Laptop | 10/20/2026 | 50 | 50
Central Warehouse | Laptop | 10/22/2026 | -20 | 30
Central Warehouse | Laptop | 10/25/2026 | -10 | 20
Central Warehouse | Wireless Mouse | 10/21/2026 | 100 | 180
Central Warehouse | Wireless Mouse | 10/23/2026 | -30 | 150
Western Warehouse | Headphones | 11/02/2026 | -10 | 10
Western Warehouse | Monitor | 10/22/2026 | 40 | 75
Western Warehouse | Monitor | 10/25/2026 | -15 | 60
OK, 8 records retrieved in 1.944ms
View:
Pages:
Edit:


Local MySQL
root@127.0.0.1:3306
Homebrew (26.7.0)
Connected
AUTO-COMMIT ON
Run
Untitled.sql
ORDER BY
#Identify stock-out dates
WITH movement_balances AS (
SELECT
movement_id,
warehouse_id,
product_id,
movement_date,
CASE
WHEN movement_type = 'IN' THEN quantity
ELSE -quantity
END AS net_change
FROM inventory_movements
),
opening_balances AS (
SELECT
i.warehouse_id,
i.product_id,
i.quantity_on_hand -
COALESCE(SUM(m.net_change), 0) AS opening_quantity
FROM inventory i
LEFT JOIN movement_balances m
ON i.warehouse_id = m.warehouse_id
Ln 720, Col 29
Output
OK, 0 records retrieved in 2.625ms



CONCLUSION

This project demonstrates how a relational database can support supply chain optimization. By implementing SQL queries, constraints, indexes, views and window functions, the system can monitor inventory, identify low-stock products, evaluate supplier performance and analyze logistics operations. The project provides a structured approach to improving inventory management and making data-driven decisions.